

TyFlow: A Type-Aware Approach to Neural Code Models

ZHECHONG HUANG, Peking University, China

ZHAO ZHANG, Peking University, China

RUYI JI, University of Michigan, USA

TINGXUAN XIA, Peking University, China

QIHAO ZHU, Peking University, China

QINXIANG CAO, Shanghai Jiao Tong University, China

ZEYU SUN, Institute of Software, Chinese Academy of Sciences, China

WIGGIN ZHOU, Tencent, China

YINGFEI XIONG*, Peking University, China

Language models have shown remarkable proficiency in code generation; nevertheless, ensuring type correctness remains a challenge. Although traditional methods, such as constrained decoding, alleviate this problem by externally rejecting untypable code, the model itself does not effectively learn type reasoning internally, which ultimately limits its overall performance.

This paper introduces TyFlow, a novel system that internalizes type reasoning within code generation to guide the model to learn the type system. The core of our approach is a novel type-guided program synthesis system that maintains an isomorphism between type derivation trees and synthesis derivation trees, enabling a new code representation based on synthesis decision sequences rather than traditional text-based token sequences. By offloading the complexity of type system learning to the representation itself, models can redirect their computational resources toward higher-level program semantics.

Our evaluation shows that TyFlow not only eliminates type errors but also significantly improves functional correctness, highlighting the importance of aligning LMs with type systems internally.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; *Formal language definitions*.

Additional Key Words and Phrases: Type Correctness, Program Synthesis, Code Generation, LLM

ACM Reference Format:

Zhechong Huang, Zhao Zhang, Ruyi Ji, Tingxuan Xia, Qihao Zhu, Qinxiang Cao, Zeyu Sun, Wiggin Zhou, and Yingfei Xiong. 2026. TyFlow: A Type-Aware Approach to Neural Code Models. *ACM Trans. Softw. Eng. Methodol.* 0, 0, Article 0 (January 2026), 46 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Recently, language models (LMs) have achieved great success in code generation [2, 19, 37, 41], where code is generated based on a natural language prompt. LMs typically treat programs as plain text, encoding them as sequences of tokens

*Corresponding author.

Authors' Contact Information: Zhechong Huang, willhuang@stu.pku.edu.cn, Peking University, Beijing, China; Zhao Zhang, zhangzhao2019@pku.edu.cn, Peking University, Beijing, China; Ruyi Ji, jiry@umich.edu, University of Michigan, Ann Arbor, USA; Tingxuan Xia, kkogoro@stu.pku.edu.cn, Peking University, Beijing, China; Qihao Zhu, Zhuqh@pku.edu.cn, Peking University, Beijing, China; Qinxiang Cao, caoqinxiang@gmail.com, Shanghai Jiao Tong University, Shanghai, China; Zeyu Sun, zeyu.zys@gmail.com, Institute of Software, Chinese Academy of Sciences, Beijing, China; Wiggin Zhou, wigginzhou@tencent.com, Tencent, Beijing, China; Yingfei Xiong, xiongyf@pku.edu.cn, Peking University, Beijing, China.

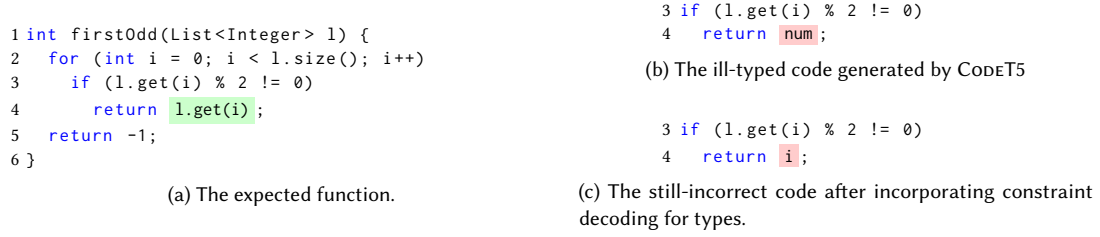


Fig. 1. A code generation task and the incorrect Java programs generated by CODET5, where the prompt comprises the function signature, input-output examples, and the description “return the first odd element from a list (-1 by default)”. We highlight the incorrect code as **red** and the expected counterpart as **green**.

and generating code by iteratively predicting the next token given the current partial program. After being trained on large corpora of code and natural language, such LMs can generate programs on demand with high accuracy.

However, language models (LMs) frequently struggle with type systems, often producing code that violates basic type constraints. Empirical studies indicate that type errors alone account for 33.6% of failed LM-generated programs [7, 31]. Notably, even cutting-edge tools powered by state-of-the-art LMs can generate code that fails to compile. For instance, an evaluation of GitHub Copilot on LeetCode revealed that 24% of its suggestions resulted in compilation errors [23], which mainly resulted from type errors [22]. This type validity problem becomes particularly pronounced in resource-limited languages or those with complex type systems [16, 17].

Ensuring type correctness is also useful beyond reducing type errors. First, the type system is essentially a static analysis system, and in theory, we can incorporate any decidable constraint into the type system. For example, Rust ~~ensured~~ensures no invalid memory access through an ownership type system. Therefore, by enriching the type system, we can enforce any decidable safety property of the code. Second, as revealed by recent studies [20, 40], even if LMs can learn to satisfy simple code constraints such as syntactic correctness given sufficient training data, by assisting LMs through external facilities for satisfying the constraints during training and inference, the overall performance of the model will further improve, possibly because the computational resources of the model can then be saved for higher-level tasks, such as algorithm design and aligning the semantics between the programs and the natural language description.

Ensuring type correctness in practice, however, remains challenging. As illustrated in Fig. 1b, which presents an ill-typed output from a widely-used and fine-tuned model CODET5 [35], the model incorrectly references an undefined variable `num`, which could have been prevented through proper type checking. Essentially, the challenge originates from the *huge gap* between the text-based token sequence representation of programs and the actual structural process of type checking.

- During training, LMs can only see sequences of syntax tokens, which provides little information on the underlying type system. From these sequences alone, LMs struggle to recover and learn the entire type system. For example, typing the function in Fig. 1a relies on the signature of the `get` method and the auto-unboxing rule in Java that allows `Integer` to be returned as `int`, but both rules are obscured within the text-based token sequence.
- During generation, LMs must ensure the type correctness of the produced code. However, type-checking the partially generated code is challenging, as it often requires complex analysis to gather necessary contextual information from the existing code. For instance, to check variable availability, the LM must identify all prior

definitions (such as `l` and `i` in Fig. 1a) and determine whether each variable remains in scope, which necessitates non-trivial global reasoning across the entire codebase. Existing LMs often struggle with such complex contextual reasoning, especially as the codebase grows in size [3, 10].

To enforce type correctness, a typical approach is *rejecting invalid generations*, as realized by a series of constrained decoding approaches [22, 36]. These approaches leverage external type checkers and reject untypable programs generated by LMs. However, such an external patch cannot bridge the gap in principle. Constrained decoding can only exclude ill-typed programs; it does not help the model learn type systems, and thus cannot improve, and may even worsen, the distribution among well-typed programs [25]. For example, in our sample task, CODET5 assigns a low probability to the expected expression (Fig. 1a), possibly because the LM is unaware of the auto-unboxing rule and thus incorrectly identifies it as ill-typed. Constrained decoding helps little with this case, as shown in Fig. 1c – it merely shifts to a still incorrect but well-typed program that avoids auto-unboxing.

To guide the model in learning the type system, a straightforward approach is to employ a reasoning model that augments the program with a type correctness proof; that is, the model first generates the program and then generates its type correctness proof as the type reasoning process, or vice versa. In this way, the type reasoning is made explicit to the model, enabling it to learn the type system of the target program. However, the separation between program generation and type reasoning introduces fundamental limitations. During training, presenting them as separate phases prevents LMs from capturing their fine-grained interactions, requiring extra effort to align them. During inference, the model must still perform global type reasoning without a localized typing context, leaving the challenge of contextual reasoning unresolved. As a result, this approach not only hinders learning efficiency but also causes inconsistencies between type reasoning and generated code. As our evaluation will later demonstrate, it does not yield satisfactory performance.

Building on these insights, we aim to propose a new synthesis system that *reconciles type systems internally with LMs*. Our approach leverages the fact that LMs are not confined to generating programs as plain text; they can be incorporated into any search-based system that constructs programs via sequences of decisions, with each decision made in a specific context. For any such system, an LM can be trained to predict the optimal decision at each step, enabling program synthesis by interacting with the LM throughout the search process.

In this way, reconciling type systems with LMs reduces to designing a synthesis system $\mathcal{S}$ that cooperates effectively with LMs while ensuring type correctness. Specifically, we expect the system $\mathcal{S}$ to satisfy the following properties, which succinctly summarize the main challenges of LM-based code generation as we discussed earlier:

- **Type Explicitness.** $\mathcal{S}$ should explicitly trace the type derivation throughout the decision process, making the type reasoning visible to the model.
- **Context Locality.** At each step, $\mathcal{S}$ should present all necessary type information within a small, bounded context window. This relieves LMs from the burden of inferring such information from lengthy contexts.
- **Derivation Vicinality.** The decisions in $\mathcal{S}$ should be represented such that each program fragment appears adjacent to its type derivation, making it easier for LMs to learn the correspondence between code and type reasoning during training.
- **Data Usability.** The decision sequences of $\mathcal{S}$ and source code should be automatically and bidirectionally convertible, enabling both the extraction of decision sequences from existing programs and the reconstruction of programs from generated decision sequences.

Our first contribution is the observation that, under constructive logic, an existential proof of type correctness inherently contains the program it proves. Formally, an existential proof Π of the following statement

$$\exists p. \text{welltyped}(p)$$

must explicitly construct a witness program p . Thus, there exists an extraction function extract such that $\text{extract}(\Pi) = p$. We leverage this insight to design a novel synthesis system and program representation that unifies program generation with type reasoning.

Based on this observation, the process of constructing an existential proof can be reformulated as a program synthesis process. The proof is built by applying typing rules step by step, where each step makes a *decision* that either applies a rule to decompose the current goal into subgoals, or instantiates a variable within the context. Since the program is embedded within the proof, this sequence of decisions simultaneously determines the program being synthesized.

We use this decision sequence as a novel program representation, which satisfies the desired properties:

- **Type Explicitness.** Constructing the decision sequence is equivalent to constructing the existential type correctness proof, making the complete type reasoning explicit throughout the process.
- **Derivation Vicinality.** The representation inherently captures both the program and its type correctness proof simultaneously, as they arise from the same construction process.

Notably, all proof rules in our system are formulated as constrained Horn clauses (see Sec. 3.1.1). This representation ~~not only subsumes the~~ subsumes prior work on syntactic correctness [20, 42] ~~but also accommodates a broader class of Turing-computable specifications. While this paper focuses on type constraints, our synthesis method is general and applicable to arbitrary logical constraints and provides a common basis for constructing synthesis rules from proof rules. The construction applies to rules in the supported CHC form with suitable unification and constraint-checking procedures. We instantiate and evaluate the framework for type correctness in Java and SuFu.~~

Our second contribution is a novel encoder-decoder model architecture tailored for our synthesis framework. During synthesis, the current proof state evolves at each step, and the model needs to extract the dynamic typing context from this state to make the next decision. Our architecture addresses this by using the encoder to process both the natural language specification and the current synthesis goal, while the decoder autoregressively generates the decision sequence. This design enables the model to access localized type information at each step without requiring global reasoning over the entire codebase, thereby satisfying the **Context Locality** property.

Our third contribution is TyFlow, an automated system for training LMs to generate well-typed programs. TyFlow takes as input (1) the language definition with its syntax, typing rules, and associated toolchain (including a parser and a type checker), and (2) a set of training tasks comprising prompts and target programs. From these inputs, TyFlow automatically instantiates the synthesis framework for the target language, providing: (1) a training component that extracts decision sequences from programs and their existential type correctness proofs, and (2) a generation component that first generates decision sequences within the synthesis system, and then extracts well-typed programs from them. This satisfies the **Data Usability** property: any well-typed program permits the extraction of its corresponding decision sequence, while any valid decision sequence can be reconstructed into a well-typed program along with its proof.

Our experimental results show that, relative to vanilla code generation, TyFlow not only eliminates untypable programs¹, but also substantially increases the number of programs that pass all unit tests. These findings indicate

¹with respect to the type system we implemented

```

data Prog = true | false | Var | Prog Prog | λStr:Type. Prog
data Type = bool | Type → Type
data Context = empty | Context, Var:Type

```

(a) The inductive data types for the syntax.

$$\begin{array}{c}
\frac{x : t \in \Gamma}{\Gamma \vdash x : t} \text{ (T-VAR)} \quad \frac{\Gamma, x : t_1 \vdash p : t_2}{\Gamma \vdash (\lambda x : t_1. p) : t_1 \rightarrow t_2} \text{ (T-ABS)} \quad \frac{\Gamma \vdash p_1 : t_1 \rightarrow t_2 \quad \Gamma \vdash p_2 : t_1}{\Gamma \vdash p_1 p_2 : t_2} \text{ (T-APP)}
\end{array}$$

(b) Selected typing rules for $\lambda^\rightarrow$. Each typing rule derives a typing judgment of the form $\Gamma \vdash p : t$, which asserts that program p has type t under context Γ .

Fig. 2. The definition of $\lambda^\rightarrow$ in TyFlow, comprising its syntax and some selected typing rules. $\dashv$

that TyFlow successfully helps the LM acquire a deeper understanding of the type system and yields a more precise distribution over correct programs.

The rest of the paper is organized as follows. Sec. 2 gives an overview of our approach. Sec. 3 introduces how we translate the typing rules into synthesis rules. Sec. 4 proves the correctness of our framework and introduces how we prepare the training data and perform synthesis within the framework. Sec. 5 introduces our neural network architecture. Sec. 6 describes the evaluation of our approach. Finally, Sec. 7 discusses related works and Sec. 8 concludes the paper.

2 Overview

This section gives an overview of our approach TyFlow, illustrating how it generates a well-typed program in the simply typed lambda calculus $\lambda^\rightarrow$ (a foundational functional programming language) [6].

2.1 Problem Definition

We formulate the type-safe code generation problem based on two primary inputs: a language specification and a dataset of training tasks.

Language definition. As shown in Fig. 2, a language in TyFlow is specified by its syntax and its type system. In this paper, we define the syntax as a set of inductive data types over several built-in base types (e.g., string, bool). Here, the syntax of $\lambda^\rightarrow$ consists of three data types (Fig. 2a): Prog and Type define programs and types in $\lambda^\rightarrow$ as the standard simply typed lambda calculus, above the base type bool; and Context specifies a context for variable types, where empty denotes the empty context, and $(\Gamma, x : t)$ extends a context Γ by binding the type t to the variable x .

To specify the type system, we require the definition of each typing rule (which defines how types are assigned to program constructs) together with an executable type checker. For clarity, we present our approach using the standard type notations of $\lambda^\rightarrow$ as listed in Fig. 2b. The type checker is expected to generate a type correctness proof (i.e., a type derivation tree, where each tree node corresponds to an application of a typing rule, like Fig. 4) for the typing judgement $\text{empty} \vdash p : t$, which asserts that program p has type t under the empty context.

Training tasks. Following the standard code generation paradigm, the objective is to generate correct programs from natural language specifications. Therefore, each training task in TyFlow is specified by a prompt paired with the expected program. Fig. 3 introduces an example task over $\lambda^\rightarrow$, and Fig. 4 illustrates the type correctness proof of the expected program generated by the type checker during type checking.

Prompt: Implement an identity function that takes a boolean input and returns the same value; then apply it to the value true.

Expected program p^* :

$(\lambda x : \text{bool}. x) \text{ true}$

Fig. 3. A training task over $\lambda^{\rightarrow}$.

$$\frac{\frac{x : \text{bool} \in \{\text{empty}, x : \text{bool}\}}{\text{empty}, x : \text{bool} \vdash x : \text{bool}} \text{ (T-VAR)} \quad \frac{\text{empty}, x : \text{bool} \vdash x : \text{bool}}{\text{empty} \vdash \lambda x : \text{bool}. x : \text{bool} \rightarrow \text{bool}} \text{ (T-Abs)} \quad \frac{\text{empty} \vdash \text{true} : \text{bool}}{\text{empty} \vdash (\lambda x : \text{bool}. x) \text{ true} : \text{bool}} \text{ (T-APP)} \quad (...)$$

Fig. 4. The type correctness proof of the expected program p^* in Fig. 3.

Table 1. Synthesis Process of TyFlow for the program in Fig. 3

Step	Current Goal	Rule Applied	Substitution	New Subgoals	Current Program
1	$\text{empty} \vdash p : t$	T-APP	$\sigma_1 = \{\Gamma_1 \mapsto \text{empty}, p \mapsto p_1 p_2, t \mapsto t_2\}$	$\text{empty} \vdash p_1 : t_1 \rightarrow t_2$ $\text{empty} \vdash p_2 : t_1$	$p_1 p_2$
2	$\text{empty} \vdash p_1 : t_1 \rightarrow t_2$	T-Abs	$\sigma_2 = \{\Gamma_2 \mapsto \text{empty}, p_1 \mapsto \lambda x_1 : t_3. p_3, \dots\}$	$\text{empty}, x_1 : t_3 \vdash p_3 : t_4$	$(\lambda x_1 : t_3. p_3) p_2$
3	$\text{empty}, x_1 : t_3 \vdash p_3 : t_4$	T-VAR	$\sigma_3 = \{\Gamma_3 \mapsto (\text{empty}, x_1 : t_3), p_3 \mapsto x_2, t_4 \mapsto t_5\}$ $\sigma_4 = \{x_1 \mapsto x, x_2 \mapsto x, \dots\}$	None	$(\lambda x : \text{bool}. x) p_2$
4	$\text{empty} \vdash p_2 : \text{bool}$	...	$\sigma_6 = \{p_2 \mapsto \text{true}, \dots\}$	None	$(\lambda x : \text{bool}. x) \text{ true}$

2.2 From Proof Construction to Program Synthesis

Given the inputs in Fig. 3, TyFlow aims to generate well-typed $\lambda^{\rightarrow}$ programs from prompts. Our core insight is that we can construct an existential type correctness proof even when the program is unknown, thereby synthesizing the program while constructing the proof. This is possible because each typing rule restricts a part of the program structure, making the structure of the proof sufficient to determine the entire program.

For example, in Fig. 4, the T-APP rule restricts the top-level operator of the program to be a function application, the T-Abs rule restricts the function term to be a lambda abstraction, and the application of T-VAR defines that the body of the lambda abstraction is a variable. In other words, **typing rules inherently serve a program synthesis role**. This observation allows us to construct a synthesis system directly grounded in typing rules, enabling simultaneous program synthesis and type reasoning.

We illustrate the synthesis process of TyFlow in detail as shown in Tab. 1, where we use purple to distinguish the unknown variables (i.e., placeholders to be filled in) in each goal, which will be instantiated later during the synthesis. For the $\lambda^{\rightarrow}$ language, TyFlow begins by specifying the synthesis goal as $\text{empty} \vdash p : t$. This synthesis goal is a typing judgment with unknown variables, and it corresponds to constructing an existential proof of $\exists p, t. \text{empty} \vdash p : t$: the unknown variables in the typing judgment are exactly the existential variables to be witnessed in the proof. The initial context empty indicates that no variables have been bound at the beginning.

During synthesis, the system incrementally applies typing rules to construct the proof, and fills in the values of p and t correspondingly. For other programming languages with different type systems, the synthesis goal can be adapted accordingly.

Step 1: applying T-APP. Given the initial synthesis goal $\text{empty} \vdash p : t$, the LM must select a typing rule to apply. Assume the rule T-APP is selected. The system then verifies that this application is admissible for the current goal,

applies the rule, and generates the corresponding subgoals (one for each premise). More concretely, the system performs the following steps in order:

- (1) First, TyFlow α -renames the rule by replacing all variables in the rule with fresh ones to avoid name conflicts. Under this renaming, T-APP takes the following form.

$$\frac{\Gamma_1 \vdash p_1 : t_1 \rightarrow t_2 \quad \Gamma_1 \vdash p_2 : t_1}{\Gamma_1 \vdash p_1 p_2 : t_2}$$

- (2) Subsequently, TyFlow attempts to unify the rule's conclusion judgment with the current synthesis goal $\text{empty} \vdash p : t$, accumulating the induced constraints as the equations shown below. If unification fails, the rule application is considered invalid and discarded.

$$\Gamma_1 = \text{empty} \quad \wedge \quad p_1 p_2 = p \quad \wedge \quad t_2 = t$$

- (3) Thereafter, TyFlow attempts to resolve these constraints via unification, yielding a substitution that maps variables to their values. If unification fails, the rule application is rejected.

$$\sigma_1 = \{\Gamma_1 \mapsto \text{empty}, \quad p \mapsto p_1 p_2, \quad t \mapsto t_2\}$$

- (4) Finally, TyFlow constructs synthesis subgoals by applying the substitution to each premise of T-APP, which has two premises corresponding to the function and the argument of the application, $\Gamma_1 \vdash p_1 : t_1 \rightarrow t_2$ and $\Gamma_1 \vdash p_2 : t_1$, and TyFlow will generate two subgoals:

$$\text{empty} \vdash p_1 : t_1 \rightarrow t_2 \quad \text{empty} \vdash p_2 : t_1$$

Each subgoal corresponds to constructing an existential sub-proof. Specifically, the first subgoal $\text{empty} \vdash p_1 : t_1 \rightarrow t_2$ corresponds to proving $\exists p_1, t_1, t_2. \text{empty} \vdash p_1 : t_1 \rightarrow t_2$, and the second subgoal corresponds to proving $\exists p_2. \text{empty} \vdash p_2 : t_1$. Once these sub-proofs are constructed with concrete witnesses, the substitution σ_1 determines the original existential variables: $p = p_1 p_2$ and $t = t_2$. TyFlow proceeds to discharge the two subgoals recursively before closing the parent goal.

Step 2: applying T-ABS. TyFlow applies the same procedure to the first subgoal $\text{empty} \vdash p_1 : t_1 \rightarrow t_2$. The LM will select the T-ABS with a high probability rule with high probability, as this goal requires an arrow type. TyFlow then α -renames the T-ABS rule:

$$\frac{\Gamma_2, x_1 : t_3 \vdash p_3 : t_4}{\Gamma_2 \vdash (\lambda x_1 : t_3. p_3) : t_3 \rightarrow t_4}$$

By performing the remaining three actions as detailed in Step 1, TyFlow again obtains a substitution and generates a new subgoal as follows.

$$\begin{aligned} \text{Substitution:} \quad \sigma_2 &= \{\Gamma_2 \mapsto \text{empty}, \quad p_1 \mapsto \lambda x_1 : t_3. p_3, \quad t_1 \mapsto t_3, \quad t_2 \mapsto t_4\} \\ \text{Subgoal:} \quad &\text{empty}, x_1 : t_3 \vdash p_3 : t_4 \end{aligned}$$

Step 3: applying T-VAR. Next, TyFlow handles the newly generated subgoal $\text{empty}, x_1 : t_3 \vdash p_3 : t_4$, assuming the LM selects the T-VAR rule. TyFlow again α -renames the rule and then obtains the substitution as shown below.

$$\begin{aligned} \text{Rule:} \quad &\frac{x_2 : t_5 \in \Gamma_3}{\Gamma_3 \vdash x_2 : t_5} \\ \text{Substitution:} \quad \sigma_3 &= \{\Gamma_3 \mapsto (\text{empty}, x_1 : t_3), \quad p_3 \mapsto x_2, \quad t_4 \mapsto t_5\} \end{aligned}$$

T-VAR differs from previous rules in that its premise involves the membership predicate $\in$, which checks whether a variable binding exists in the context. Rather than generating a new subgoal, TyFlow directly invokes the predicate $\in$ to assess the validity of its arguments. However, there are still unknown variables in both the premise and the synthesis goal even after applying the substitution σ_3 :

Premise: $x_2 : t_5 \in (\text{empty}, x_1 : t_3)$
 Goal: $\text{empty}, x_1 : t_3 \vdash x_2 : t_5$

The unknown variables in the premise prevent TyFlow from evaluating the predicate. To address this issue, TyFlow will query the LM to provide an assignment to all such variables, such as follows:

$$\sigma_4 = \{x_1 \mapsto x, x_2 \mapsto x, t_3 \mapsto \text{bool}, t_5 \mapsto \text{bool}\} \quad (1)$$

By further applying σ_4 , the premise becomes $x : \text{bool} \in (\text{empty}, x : \text{bool})$, which is completely known; then, TyFlow can evaluate the predicate and confirm that it is satisfied. At this time, no new subgoal is generated, so the synthesis process for this branch is complete, where the result is the composition of σ_3 and σ_4 (i.e., applying both substitutions in sequence):

$$\sigma_4\sigma_3 = \{t_3 \mapsto \text{empty}, x : \text{bool}, p_3 \mapsto x, t_4 \mapsto \text{bool}, x_1 \mapsto x, x_2 \mapsto x, t_3 \mapsto \text{bool}, t_5 \mapsto \text{bool}\}$$

Step 4: returning from subgoals. After finishing a subgoal, TyFlow will apply the obtained substitution to all remaining subgoals to synchronize the progress; and once all subgoals are closed, TyFlow will compose all obtained substitutions as the final substitution for the synthesis goal.

In this example, the substitution $\sigma_4\sigma_3$ obtained in Step 3 is further composed with the substitution σ_2 from Step 2, and the resulting composition $\sigma_5 = \sigma_4\sigma_3\sigma_2$ is returned as the result of the subgoal $\text{empty} \vdash p_1 : t_1 \rightarrow t_2$ in Step 2. This yields a concretized and verified typing judgment, $\text{empty} \vdash \lambda x : \text{bool}. x : \text{bool} \rightarrow \text{bool}$. Since there are two subgoals generated in Step 1, TyFlow applies the composed substitution σ_5 to the second subgoal, $\text{empty} \vdash p_2 : t_1$, yielding

$$\text{empty} \vdash p_2 : \text{bool}.$$

TyFlow then applies the same procedure to this subgoal, producing the substitution $\sigma_6 = \{p_2 \mapsto \text{true} \dots\}$. This new substitution is then composed with the previous σ_1 and σ_5 to yield $\sigma_7 = \sigma_6\sigma_5\sigma_1$, which is applied to the original synthesis goal $\text{empty} \vdash p : t$, giving the final result:

$$\text{empty} \vdash (\lambda x : \text{bool}. x) \text{ true} : \text{bool}$$

At this point, we have successfully constructed an existential proof of $\exists p, t. \text{empty} \vdash p : t$, with the existential variables instantiated as $p = (\lambda x : \text{bool}. x) \text{ true}$ and $t = \text{bool}$. The synthesized program p is thus guaranteed to be well-typed.

Performing program synthesis via existential proof construction embodies our two key principles: **Type Explicitness** and **Context Locality**.

- **Type Explicitness.** Each synthesis step corresponds to applying a typing rule, which directly mirrors the structure of the type correctness proof. As shown in the example, the sequence of rule applications (T-APP, T-ABS, T-VAR, etc.) constructs a complete proof tree, making the type reasoning explicit throughout the synthesis process.
- **Context Locality.** At each step, the synthesis goal is a partially instantiated typing judgment that contains the current typing context. For instance, in Step 2, the goal $\text{empty} \vdash p_1 : t_1 \rightarrow t_2$ explicitly provides the context

$$\begin{array}{c}
\frac{\Gamma \vdash p_1 : t_1 \rightarrow t_2 \quad \Gamma \vdash p_2 : t_1}{\Gamma \vdash p_1 p_2 : t_2} \text{ (T-APP)} \quad \frac{\text{① } \begin{array}{c} \Gamma, p_1, p_2, t_1, t_2 \notin \text{FV}(\Gamma^\circ, p^\circ, T^\circ) \\ \text{Unify} \left(\begin{array}{c} \Gamma = \Gamma^\circ \\ p_1 p_2 = p^\circ \\ t_2 = T^\circ \end{array} \right) = \sigma \end{array} \quad \begin{array}{c} \text{② } \sigma(\Gamma) \vdash \sigma(p_1) : \sigma(t_1 \rightarrow t_2) \rightsquigarrow \sigma_1 \\ \text{③ } \sigma' = \sigma_1 \sigma \\ \sigma'(\Gamma) \vdash \sigma'(p_2) : \sigma'(t_1) \rightsquigarrow \sigma_2 \end{array}}{\Gamma^\circ \vdash p^\circ : T^\circ \rightsquigarrow \sigma_2 \sigma' |_{\text{FV}(\Gamma^\circ, p^\circ, T^\circ)}} \text{ (S-APP)}
\end{array}$$

Fig. 5. The typing rule T-APP and its corresponding synthesis rule S-APP, where $\text{FV}(\Gamma^\circ, p^\circ, T^\circ)$ denotes the unknown variables in the incomplete typing judgment. S-APP will return assignments only for these variables.

empty and indicates that the program p_1 to be synthesized must have an arrow type $t_1 \rightarrow t_2$. This local type information guides the LM to select the T-Abs rule, which produces a lambda abstraction.

The $\lambda^\rightarrow$ language above serves for exposition: it is deliberately minimal, containing only the base type `bool`, function types, and explicitly annotated, monomorphic bindings. Real languages additionally involve subtyping, polymorphism, type inference, overloading resolution, and mutable state, none of which $\lambda^\rightarrow$ exercises. The evaluation (Sec. 6) therefore runs TyFlow on Java and SuFu instances with fuller type systems; the scope of the underlying rule framework is stated in Sec. 3, and how the approach would extend to richer type systems is discussed in Sec. 6.3.

2.3 Translating Typing Rules to Synthesis Rules

In summary of Sec. 2.2, TyFlow extends typing rules for synthesis with the two design choices:

- When applying a rule, instead of pattern matching, TyFlow introduces rule parameters as fresh variables and solves the constraints between variables via unification.
- Whenever an executable predicate requires the value of an unknown variable, TyFlow acquires this value directly from the LM.

With these ideas, typing rules can be restated as formal synthesis rules in a mechanical way, which will be elaborated in Sec. 3. Here we shall illustrate this process with the T-APP rule as in Fig. 5.

To synthesize programs in $\lambda^\rightarrow$, a synthesis judgment takes the form:

$$\Gamma^\circ \vdash p^\circ : t^\circ \rightsquigarrow \sigma$$

- The left side is an incomplete typing judgment, where Γ° , p° , and t° represent a (possibly) incomplete context, program, and type, respectively. For example, t° can be an unknown variable t , a partially known type `bool  $\rightarrow$   $t$` , or a fully known type `bool  $\rightarrow$  bool`.
- The right side is an assignment σ to all unknown variables on the left side.

This judgment means that **the left-hand side typing judgment can be properly completed by the right-hand side assignment**; in other words, after applying σ as a substitution, $\sigma(\Gamma^\circ) \vdash \sigma(p^\circ) : \sigma(t^\circ)$ will be a valid typing judgment in the type system of $\lambda^\rightarrow$. From the perspective of proof construction, the synthesis judgment corresponds to constructing an existential proof: given a proof goal with unknown variables, the synthesis process finds witnesses (i.e., concrete values for these variables) and proves the validity of the goal.

As marked in Fig. 5, the premises of rule S-APP can be divided into three parts.

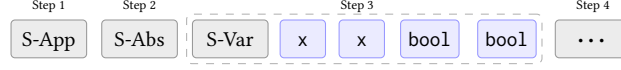


Fig. 6. Synthesis decision sequence for $(\lambda x : \text{bool}. x) \text{ true}$: synthesis rules and variable assignments.

- Part 1 introduces a fresh variable for each parameter in the typing rule T-APP, unifies the conclusion of T-APP with the current incomplete typing judgment (i.e., $\Gamma \vdash p_1 p_2 : t$ with $\Gamma^\circ \vdash p^\circ : t^\circ$) and thus extracts the relation between variables as a substitution σ .
- Parts 2 and 3 are introduced regarding the two premises in the T-APP rule. Part 2 completes the first premise $\Gamma \vdash p_1 : t_1 \rightarrow t_2$, resulting in an assignment σ_1 . Then, Part 3 combines the known information by composing σ and σ_1 into σ' , and further completes the second premise $\Gamma \vdash p_2 : t_1$. In general, this process repeats for all premises until each is completed, and any failure to discharge a premise causes the entire rule application to fail.

To get the result, TyFlow composes the two obtained substitutions σ_2 and σ' . Note that this composition involves not only variables in $\Gamma^\circ \vdash p^\circ : t^\circ$ but also local variables such as Γ and p_1 . Therefore, S-APP will exclude these local variables and return only values of interest.

This translation establishes a direct correspondence between typing rules and synthesis rules. The core insight is that when the program is unknown, we can apply synthesis rules to construct an existential proof, which simultaneously discovers the program and proves its type correctness. The resulting existential proof has the same structure as the type correctness proof of the discovered program (like the one depicted in Fig. 4).

2.4 Interacting with the Language Model

Having defined the synthesis rules, we now describe how the LM interacts with the synthesis system.

LM Queries. The synthesis rules form a search system for program synthesis, where the system issues queries to the LM and recursively discharges synthesis subgoals based on the decisions the LM makes. As discussed in Sec. 2.2, at each synthesis step, the system raises two types of queries: to select an appropriate synthesis rule, and to generate variable assignments for predicates. In TyFlow, we resolve both queries by prompting the LM with the following information:

- (1) the natural language prompt of the overall synthesis task
- (2) the sequence of synthesis decisions generated so far
- (3) the current synthesis goal

To simplify the interaction, we do not distinguish the two types of queries and assume that the LM can learn to infer what information is needed in each step from the context. Fig. 6 illustrates the expected decision sequence for our sample task. Let us consider the first three steps:

- In Step 1, the LM receives the task prompt, an empty decision sequence, and the overall synthesis goal $\text{empty} \vdash p : t$. It predicts the synthesis rule S-APP with no other output, as no variable assignment is needed in this rule application.
- In Step 2, the case is similar. The LM predicts only S-Abs with no other assignment.
- In Step 3, the decision sequence becomes [S-APP, S-Abs] and the subgoal becomes $\text{empty}, x_1 : t_3 \vdash p_3 : t_4$. Here, the LM predicts the S-VAR rule, and meanwhile, generates all four assignments (Formula 1) required by this rule.

As shown in Fig. 6, the synthesis rules and variable assignments together form a synthesis decision sequence, which serves as a novel program representation in TyFlow. Unlike traditional representations that treat programs as flat

sequences of syntactic tokens (e.g., $[\lambda, x, :, \text{bool}, \dots]$), our representation is structured around the type correctness proof. Each token corresponds to a step in the proof construction process, directly capturing the hierarchical structure of type reasoning (as in Fig. 4). This enables the LM to learn the type system explicitly rather than attempting to reconstruct it from flat token sequences. Since this sequence represents a program synthesis process that determines both the resulting program and its type correctness proof, this representation embodies our **Derivation Vicinality** principle.

To support this proof-guided synthesis, we design a novel model architecture with dual-encoding: the encoder processes both the static natural language specification and the dynamically evolving synthesis goal at each step, providing rich typing context to the decoder. Details of this architecture are presented in Sec. 5.

Training. Since our representation differs from traditional token sequences, the LM requires fine-tuning to generate synthesis decision sequences. Our representation also offers **Data Usability**, as it is compatible with existing code generation datasets for the target language. Given a corpus of programs, we construct their type correctness proofs via the type checker, convert the proofs into synthesis decision sequences using the correspondence between typing rules and synthesis rules, and obtain a training set in TyFlow’s representation.

3 Translation from Typing Rules to Synthesis Rules

In this section, we introduce the necessary preliminaries on constrained Horn clauses and unification, [followed by present](#) the user-provided specifications of the object language, [including terms, typing rules \(including terms and typing rules\)](#), and describe how typing rules are systematically translated into synthesis rules.

3.1 Preliminaries

3.1.1 Constrained Horn Clauses. Constrained Horn clauses (CHCs) are a fragment of first-order logic with applications to program verification and synthesis [12]. A constrained Horn clause, in this setting, is a universally quantified implication of the canonical form:

$$\frac{P_1(\bar{x}_1) \quad P_2(\bar{x}_2) \quad \dots \quad P_n(\bar{x}_n) \quad \phi(\bar{y})}{P(\bar{x}_0)} \text{ (CHC)}$$

Fig. 7. General form of a constrained Horn clause.

Each clause consists of zero or more premises, where each premise is a judgment formed by applying an uninterpreted predicate P_i to terms $\bar{x}_i$. It also includes a constraint $\phi(\bar{y})$, a computable function over terms $\bar{y}$, and concludes with a single judgment $P(\bar{x}_0)$. This rule states that the judgment $P(\bar{x}_0)$ can be derived whenever all the premises $P_i(\bar{x}_i)$ are derivable and the constraint $\phi(\bar{y})$ holds. Without loss of generality, we can assume that every variable of terms in $\bar{y}$ occurs among a term in the premises or conclusion, namely $FV(\bar{y}) \subseteq \bigcup_{i=0}^n FV(\bar{x}_i)$, which ensures that the constraints are always deterministic and checkable given the premises and conclusion.

Example 3.1. Consider the typing rule T-APP for function application in $\lambda \rightarrow$:

$$\frac{\Gamma \vdash p_1 : t_1 \rightarrow t_2 \quad \Gamma \vdash p_2 : t_1}{\Gamma \vdash p_1 p_2 : t_2} \text{ (T-App)}$$

In CHC terminology: the predicates P_1 and P_2 are both the typing judgment predicate $\cdot \vdash \cdot : \cdot$; the terms $\bar{x}_1 = (\Gamma, p_1, t_1 \rightarrow t_2)$ and $\bar{x}_2 = (\Gamma, p_2, t_1)$ are the arguments to the premises; the conclusion $P(\bar{x}_0)$ is $\Gamma \vdash p_1 p_2 : t_2$ with $\bar{x}_0 = (\Gamma, p_1 p_2, t_2)$; and the constraint ϕ is implicitly *True*.

It is well known that any Turing-computable function can be encoded using Horn clauses (HCs) [32]. CHCs extend HCs by adding constraints $\phi(\bar{y})$, which allow expressing computable conditions (e.g., arithmetic comparisons, equality checks) directly within the clause. When all constraints are interpreted as *True*, CHCs collapse to ordinary HCs. Consequently, CHCs are at least as expressive as HCs, and any Turing-computable function can be expressed using CHCs, ~~which ensures the expressiveness of our framework.~~

3.1.2 Unification. Unification is a fundamental operation in logic and computer science that finds substitutions to make two expressions structurally identical. Formally, given two terms s and t , a *unifier* is a substitution σ such that $\sigma(s) = \sigma(t)$. When a unifier exists, the two terms are said to be *unifiable*. Among all possible unifiers, the *most general unifier* (MGU) provides the least restrictive solution: any other unifier can be obtained by further instantiating the MGU.

Example 3.2. Consider unifying an incomplete typing judgment $\text{empty} \vdash p_1 : t_1 \rightarrow t_2$ with another judgment $\Gamma_2 \vdash (\lambda x_1 : t_3. p_3) : t_3 \rightarrow t_4$. The unification process matches corresponding positions: empty with Γ_2 , p_1 with $\lambda x_1 : t_3. p_3$, and $t_1 \rightarrow t_2$ with $t_3 \rightarrow t_4$. This yields the MGU:

$$\sigma = \{\Gamma_2 \mapsto \text{empty}, \quad p_1 \mapsto \lambda x_1 : t_3. p_3, \quad t_1 \mapsto t_3, \quad t_2 \mapsto t_4\}$$

After applying σ , both terms become identical: $\text{empty} \vdash (\lambda x_1 : t_3. p_3) : t_3 \rightarrow t_4$.

Note that unification is symmetric: it can bind variables from either term. Here, Γ_2 is bound to a concrete value empty , while t_1 and t_2 are bound to other variables t_3 and t_4 . The MGU preserves maximum generality by not over-constraining variables. For instance, the substitution $\{\Gamma_2 \mapsto \text{empty}, p_1 \mapsto \lambda x_1 : t_3. p_3, t_1 \mapsto t_3, t_2 \mapsto t_4, t_3 \mapsto \text{bool}\}$ is also a valid unifier, but it is not the MGU because it unnecessarily constrains t_3 to bool .

In our synthesis framework, unification enables typing rules to be applied by aligning the schematic parameters in a rule’s conclusion with the current synthesis goal. When a typing rule is selected, unification extracts the relationship between the goal’s unknowns and the rule’s parameters, propagating known information and establishing constraints between unknowns. In this paper, we restrict ourselves to *first-order unification*, where variables range only over terms and not over functions or predicates. First-order unification ~~is decidable and admits efficient algorithmic solutions [28], making it suitable for our synthesis framework.~~ always terminates. It either rejects two terms or returns a most-general unifier that is unique up to variable renaming [28]. These guarantees let TyFlow apply a selected typing rule and update its remaining premises.

Supporting higher-order functions does not by itself require higher-order unification. The latter is needed only when an unknown to be solved is itself a function, type constructor, or predicate. Most languages never create such problems: their unknowns are ordinary types, and first-order unification suffices for applying their typing rules. The few languages that do, such as dependent-type systems, can be supported through a decidable fragment with principal unifiers, such as higher-order patterns, by using a different unifier, since general higher-order unification is undecidable [9]

~

3.2 Terms and Typing Rules

3.2.1 Terms. We begin by defining the object language through a structured set of terms. A *term* is a symbolic expression that represents programs, types, contexts, or other syntactic entities in the object language. Each term type is specified inductively: terms are constructed from constants, variables, and composite structures via constructors. For any given

term type $\mathcal{T}$, the syntax conforms to the general form:

$$\mathcal{T} ::= c \mid v \mid k(\mathcal{T}_1, \dots, \mathcal{T}_n)$$

where c ranges over constants $\mathcal{C}_{\mathcal{T}}$, v ranges over the global set of variables $\mathcal{V}$, and k ranges over constructor symbols $\mathcal{K}_{\mathcal{T}}$ of specified arity and parameter term types.

The variable set $\mathcal{V}$ consists of symbolic names serving as placeholders to be instantiated by terms. In contrast, the sets of constants $\mathcal{C}_{\mathcal{T}}$ and constructors $\mathcal{K}_{\mathcal{T}}$ are determined by user-defined inductive type declarations. Built-in types such as `Int` and `String` are provided following conventional programming language syntax.

Example 3.3. Consider the following syntax for programs in $\lambda^{\rightarrow}$:

$$\text{Prog} ::= \text{true} \mid \text{false} \mid \text{var}(\text{String}) \mid \text{app}(\text{Prog}, \text{Prog}) \mid \text{abs}(\text{String}, \text{Type}, \text{Prog})$$

This yields the following instantiations of the general term pattern:

- **Constants:** $\mathcal{C}_{\text{Prog}} = \{\text{true}, \text{false}\}$
- **Constructors:** $\mathcal{K}_{\text{Prog}} = \{\text{var} : (\text{String}), \text{app} : (\text{Prog}, \text{Prog}), \text{abs} : (\text{String}, \text{Type}, \text{Prog})\}$

Using this syntax, the program $\lambda x:\text{bool}. x$ is represented as the term $\text{abs}("x", \text{bool}, \text{var}("x"))$. This term is constructed by applying the constructor `abs` to three arguments: the string `"x"`, the type `bool`, and the subterm `var("x")`.

3.2.2 Categories of terms and substitutions. In general, terms may contain known and unknown components, so in the remainder of this paper, we do not explicitly annotate terms as in [Sec. 2.3](#). Below, we provide definitions for key concepts that will be used throughout the subsequent sections.

- (1) **Ground terms** are terms constructed exclusively from constants and constructors, containing no variables. They are fully specified expressions that require no further instantiation.
- (2) A **substitution** is a mapping $\sigma : \mathcal{V}_0 \rightarrow \mathcal{T}$, where $\mathcal{V}_0$ is a subset of variables $\mathcal{V}$, and $\mathcal{T}$ is the set of all terms. A substitution replaces each variable $v \in \mathcal{V}_0$ with a term $\sigma(v) \in \mathcal{T}$.
The **composition** of two substitutions σ_i and σ_j , denoted $\sigma_i \sigma_j$, is defined such that for any variable $v \in \mathcal{V}$, $(\sigma_i \sigma_j)(v) = \sigma_i(\sigma_j(v))$. This allows multiple substitutions to be applied sequentially. A substitution σ can also operate on a list of terms $\bar{t} = [t_1, t_2, \dots, t_n]$, yielding a new list $\sigma(\bar{t}) = [\sigma(t_1), \sigma(t_2), \dots, \sigma(t_n)]$ where σ is applied to each term individually.
- (3) An **assignment** is a special case of a substitution whose codomain ranges over ground terms. Assignments serve as the ultimate objective of the synthesis, as we require that the typing judgments, as synthesis goals, can be instantiated with no remaining uncertainties.

3.2.3 Typing Rules and Type Derivation Trees. We formalize the typing rules of the object language as CHCs for their expressiveness and standard structure, where each typing rule is provided by the user as a CHC shown in [Fig. 7](#). The user should also provide at least one typing rule whose conclusion is *well-typed* (p), the top-level judgment asserting that program p is well-typed. For example, in $\lambda^{\rightarrow}$, this judgment can be defined as $\exists t. \text{empty} \vdash p : t$, meaning that the program p is well-typed under the empty context.

A judgment $P(\bar{t})$, where $\bar{t}$ are ground terms, is *derivable* from a given set of typing rules if there exists a finite derivation tree whose root is labeled with $P(\bar{t})$, each internal node corresponds to the application of a typing rule, and all constraints of typing rules are satisfied. The type derivation tree is precisely this tree structure that witnesses the derivability of a typing judgment. From a logical perspective, a type derivation tree is a **type correctness proof**: it

proves that the typing judgment at the root holds. This correspondence between derivation trees and proofs is central to our approach. Later, we will show that a synthesis derivation tree corresponds to constructing an **existential type correctness proof** when the program is unknown: the synthesis process simultaneously discovers the program and constructs a proof of its type correctness.

More formally, a *type derivation tree* for a typing judgment $P(\bar{t})$ is defined as: (1) the root node is labeled with $P(\bar{t})$; (2) each node is labeled with a typing judgment $P_i(\bar{t}_i)$, where all terms $\bar{t}_i$ are ground terms, and annotated with the typing rule **which that** derives $P_i(\bar{t}_i)$ under **a** certain instantiation; (3) the children of each internal node correspond to the premises of the applied typing rule; and (4) for each applied typing rule, all associated constraints are satisfied under the instantiation.

For example, Fig. 4 shows a type derivation tree for the program $(\lambda x:\text{bool}.x)$ true, which proves that this program has type bool under the empty context. Consider the following subtree for the subterm $\lambda x:\text{bool}.x$:

$$\frac{\frac{x : \text{bool} \in x:\text{bool}}{x:\text{bool} \vdash x : \text{bool}} \text{ (T-Var)}}{\text{empty} \vdash \lambda x:\text{bool}.x : \text{bool} \rightarrow \text{bool}} \text{ (T-Abs)}$$

In this subtree, the root node is labeled with the typing judgment $\text{empty} \vdash \lambda x:\text{bool}.x : \text{bool} \rightarrow \text{bool}$ and annotated with the typing rule T-Abs. Its child node is labeled with $x:\text{bool} \vdash x : \text{bool}$ and annotated with T-Var, which has a constraint $x : \text{bool} \in x:\text{bool}$ that checks whether the variable x with type bool exists in the context.

The existence of a type derivation tree for *well-typed* (p) establishes that the program p is well-typed. To enable this verification, there should be a type checker in the object language toolchain that constructs the type derivation tree for *well-typed* (p) given any well-typed program p .

3.3 Synthesis rules

3.3.1 Synthesis Judgments and Synthesis Goals. As outlined in the overview, when some components of a typing judgment are unknown (e.g., the program to be synthesized), typing rules can be reformulated as synthesis rules to handle such uncertainties. A typing judgment $R(\bar{t})$ is converted into a *synthesis judgment* $R(\bar{t}) \rightsquigarrow \theta$, where $R(\bar{t})$ denotes the *synthesis goal*, and θ is the assignment synthesized for the goal. Semantically, $R(\bar{t}) \rightsquigarrow \theta$ is derivable iff, after applying θ to $\bar{t}$, all terms in $\theta(\bar{t})$ are ground and the typing judgment $R(\theta(\bar{t}))$ is derivable.

3.3.2 Translation from Typing Rules into Synthesis Rules.

$$\frac{\begin{array}{l} P_1(\sigma_0\sigma(\bar{x}_1)) \rightsquigarrow \sigma_1 \\ P_2(\sigma_1\sigma_0\sigma(\bar{x}_2)) \rightsquigarrow \sigma_2 \\ \vdots \\ \text{Unify}(\bar{x} = \bar{x}_0) = \sigma \\ \text{Acquire}(\sigma(\bar{x}_f)) = \sigma_0 \end{array} \quad \begin{array}{l} P_n(\sigma_{n-1} \dots \sigma_0\sigma(\bar{x}_n)) \rightsquigarrow \sigma_n \\ \phi(\sigma_n \dots \sigma_0\sigma(\bar{y})) \end{array}}{P(\bar{x}) \rightsquigarrow \sigma_n \dots \sigma_0\sigma|_{\text{FV}(\bar{x})}} \text{ (S-Rule)}$$

Fig. 8. Normal form of the synthesis rules

A typing rule in Fig. 7 can be translated into a synthesis rule in Fig. 8 in the following manner:

- (1) **Conclusion translation.** The conclusion $P(\bar{x}_0)$ of the typing rule is replaced by a synthesis judgment $P(\bar{x}) \rightsquigarrow \sigma_n \dots \sigma_0\sigma|_{\text{FV}(\bar{x})}$ (with the same predicate P), where $\bar{x}$ is a sequence of newly introduced variables and can be instantiated with respect to the synthesis goal. $\text{FV}(\bar{x})$ denotes the set of variables in $\bar{x}$, which are the only variables that need to be synthesized. $\sigma_n \dots \sigma_0\sigma|_{\text{FV}(\bar{x})}$ indicates that the final assignment is restricted to only those variables in $\bar{x}$.

- (2) **Adding unification.** The unification step is added at the beginning of the synthesis rule, which unifies the variables $\bar{x}$ in the synthesis goal with the variables $\bar{x}_0$ in the conclusion of the typing judgment, yielding a substitution σ that is the MGU of the unification problem.
- (3) **Adding Free variables acquisition.** The variables appearing in $\bar{x}_0$ but not in any of the $\bar{x}_i$ are collected into $\bar{x}_f$, defined as $\bar{x}_f = \text{list}(\text{FV}(\bar{x}_0) \setminus \bigcup_{i=1}^n \text{FV}(\bar{x}_i))$. We refer to $\bar{x}_f$ as the *free variables* of the synthesis rule: they may not be bound by any premise even after unification, and thus should be acquired from external sources, yielding the assignment σ_0 .
- (4) **Premises translation.** Each premise $P_i(\bar{x}_i)$ is replaced by $P_i(\sigma_{i-1} \cdots \sigma_0 \sigma(\bar{x}_i)) \rightsquigarrow \sigma_i$, where $\sigma_{i-1} \cdots \sigma_0 \sigma(\bar{x}_i)$ instantiates the original terms $\bar{x}_i$ with substitutions from previous steps, and σ_i is the assignment synthesized for the subgoal $P_i(\sigma_{i-1} \cdots \sigma_0 \sigma(\bar{x}_i))$ ($i \geq 1$).
- (5) **Constraint translation.** The constraint $\phi(\bar{y})$ is translated into $\phi(\sigma_n \cdots \sigma_0 \sigma(\bar{y}))$, which checks whether it holds under the assignment $\sigma_n \cdots \sigma_0 \sigma$.

Example 3.4. For example, if we have the following typing rule:

$$\frac{P_1(x_1) \quad P_2(x_2, c(x_1, x_3)) \quad \phi(x_0, x_3)}{P(x_0, c(x_1, x_2))} \text{ (T-Rule1)}$$

We can translate it into the following synthesis rule:

$$\frac{\begin{array}{l} \text{Unify}(s_0 = x_0, s_1 = c(x_1, x_2)) = \sigma \quad P_1(\sigma_0 \sigma(x_1)) \rightsquigarrow \sigma_1 \\ \text{Acquire}(\sigma(x_0)) = \sigma_0 \quad P_2(\sigma_1 \sigma_0 \sigma(x_2, c(x_1, x_3))) \rightsquigarrow \sigma_2 \quad \phi(\sigma_2 \sigma_1 \sigma_0 \sigma(x_0, x_3)) \end{array}}{P(s_0, s_1) \rightsquigarrow \sigma_2 \sigma_1 \sigma_0 \sigma|_{\text{FV}(s_0, s_1)}} \text{ (S-Rule1)}$$

LEMMA 3.5. *There is a one-to-one correspondence between typing rules and synthesis rules.*

PROOF. The complete proof can be found in Appendix A.1. □

4 Synthesis System

Fig. 9 presents the architecture of TyFlow’s system, which constructs synthesis derivation trees from which programs are extracted. The construction is achieved in Alg. 1 through a series of queries (Sec. 4.1.3), which can be resolved through two approaches corresponding to the dual requirements of our system: code generation (Sec. 4.2.1) and training data preparation (Sec. 4.2.2).

The central claim of this section is that constructing a synthesis derivation tree, which is the core of TyFlow’s framework, is equivalent to constructing an existential type correctness proof. A key insight underlying this equivalence is the isomorphism between type derivation trees and synthesis derivation trees (Sec. 4.3). This isomorphism ensures that the constructed synthesis derivation trees maintain the logical flow and dependencies inherent in the type-level reasoning, so that we can construct synthesis derivation trees to represent well-typed programs (Sec. 4.4).

4.1 Construction of Synthesis Derivation Trees

In this subsection, we will first describe the application of synthesis rules (Sec. 4.1.1), then introduce the concept of synthesis derivation trees (Sec. 4.1.2), and finally present the algorithmic construction of synthesis derivation trees, where it is realized by a sequence of query resolutions (Sec. 4.1.3).

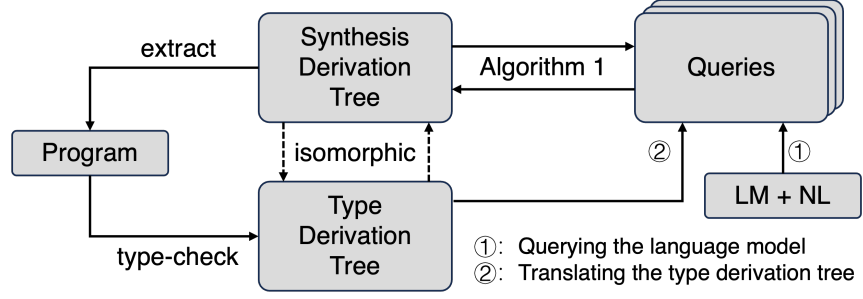


Fig. 9. Meta-system architecture of TyFlow.

4.1.1 Synthesis Rule Application. Each synthesis rule as in Fig. 8 specifies how to start from a partially instantiated typing judgment $P(\bar{x})$, search for an assignment σ to all variables in $\bar{x}$, then derive $P(\sigma(\bar{x}))$ using the corresponding typing rule. This procedure can be detailed as follows:

- (1) **Unification Step:** First unify the terms in the synthesis goal $P(\bar{x})$ with the terms in the conclusion of the typing rule, yielding the MGU σ . Once the unification is done, the other terms $(\bar{x}_i, \bar{x}_f, \text{ and } \bar{y})$ should also be instantiated with σ to match the synthesis goal. ~~Or~~ If the unification fails, ~~in which case~~ the synthesis goal is not derivable using this synthesis rule.
- (2) **Acquisition Step:** Next acquire the assignment for all free variables $\bar{x}_f$ in the synthesis ~~rules (after rule (after~~ being instantiated by σ) from external sources. TyFlow can infer the term type of each variable to be synthesized based on the definitions of predicates and terms. The generated assignments must conform to the inductively defined syntax described in Sec. 3.2.1; if any constant or constructor does not match the required term type, the acquisition step fails.
- (3) **Sub-Derivation Steps:** Then, synthesize assignments for all subgoals in sequence. For each subgoal $P_i(\sigma_{i-1} \dots \sigma_0 \sigma(\bar{x}_i)) \rightsquigarrow \sigma_i P_i(\sigma_{i-1} \dots \sigma_0 \sigma(\bar{x}_i)) \rightsquigarrow \sigma_i$, begin with an instantiation for variables in $\bar{x}_i$ with the substitutions $\sigma_{i-1}, \dots, \sigma_0, \sigma$ in previous steps. This refines the subgoal and propagates synthesis information from earlier subgoals. Subsequently, synthesize the assignment σ_i for the current subgoal by recursively applying synthesis rules.
- (4) **Constraint Checking Step:** After all synthesis subgoals have been solved, TyFlow checks whether the constraint $\phi(\bar{y})$ is satisfied under the combined assignment $\sigma_n \dots \sigma_0 \sigma$.
- (5) **Assignment Collection Step:** In the final step, TyFlow composes sall substitutions obtained during synthesis to form $\sigma_c = \sigma_n \dots \sigma_0 \sigma$, and restricts sthis assignment to only the variables occurring in $\bar{x}$, yielding the overall assignment $\theta = \sigma_c|_{FV(\bar{x})}$.

In steps (3) and (5), we assert without proof that the composed substitution $\sigma_c = \sigma_n \dots \sigma_0 \sigma$ and the final collected θ are assignments. Specifically, σ_c maps all variables in $\bar{x}$ and $\bar{y}$ to ground terms. This property can be formally established by induction on the structure of the synthesis derivation.

LEMMA 4.1. *For any successful application of the synthesis rule in Fig. 8, the substitution $\sigma_c = \sigma_n \dots \sigma_0 \sigma$ is an assignment, in the sense that $\forall i, \sigma_c(\bar{x}_i), \sigma_c(\bar{x})$, and $\sigma_c(\bar{y})$ are all ground terms.*

PROOF. The complete proof can be found in Appendix A.2. □

Algorithm 1: Synthesis Derivation Tree Construction

Input: A synthesis goal $P(\bar{t})$
Output: A synthesis derivation tree T whose root node is labeled $P(\bar{t}) \rightsquigarrow \theta$

```

1 Function GenSynthTree( $P(\bar{t})$ ):
2   Select a synthesis rule  $R$  with conclusion  $P(\bar{x}) \rightsquigarrow \dots$ ;           // Suppose  $R$  is as Fig. 8
3   Instantiate  $\bar{x}$  with  $\bar{t}$  in  $R$ ;
4    $\sigma \leftarrow$  MGU from the unification step of  $R$ ;
5   if  $\sigma = \perp$  then return  $\perp$ ;
6    $\sigma_0 \leftarrow$  AcquireFreeVariableAssignment( $\sigma(\bar{x}_f)$ );
7   if  $\sigma_0 = \perp$  then return  $\perp$ ;
8   foreach subgoal  $P_i(\sigma_{i-1} \dots \sigma_0 \sigma(\bar{x}_i))$  of  $R$  do
9      $T_i \leftarrow$  GenSynthTree( $P_i(\sigma_{i-1} \dots \sigma_0 \sigma(\bar{x}_i))$ );
10    if  $T_i = \perp$  then return  $\perp$ ;
11     $\sigma_i \leftarrow$  assignment from the root node of  $T_i$ ;
12  if  $\phi(\sigma_n \dots \sigma_0 \sigma(\bar{y})) = \perp$  then return  $\perp$ ;
13   $\theta \leftarrow \sigma_n \dots \sigma_0 \sigma|_{FV(\bar{t})}$ ;
14  return MakeTreeNode( $P(\bar{t}) \rightsquigarrow \theta, R, \sigma_0, T_1, \dots, T_n$ )

```

4.1.2 Synthesis Derivation Trees. To formalize the structure of synthesis derivations, we introduce the concept of synthesis derivation trees in analogy to type derivation trees in type systems.

A *synthesis derivation tree* is a rooted tree that witnesses the derivability of a synthesis judgment through the application of synthesis rules. More formally, a synthesis derivation tree for a synthesis judgment $P(\bar{t}) \rightsquigarrow \theta$ is a tree where: (1) the root node is labeled with $P(\bar{t}) \rightsquigarrow \theta$; (2) each node is labeled with a synthesis judgment and annotated with both the synthesis rule used and the assignment for its free variables; (3) the children of each internal node correspond to the subgoals of the annotated synthesis rule; (4) for each synthesis rule, all steps including unification, assignment acquisition, subgoals synthesis and constraint checking succeed.

By Lemma 4.1, the existence of a synthesis derivation tree for a synthesis judgment $P(\bar{t}) \rightsquigarrow \theta$ guarantees that there is an assignment θ satisfying the synthesis goal $P(\bar{t})$, that is, θ maps all variables to ground terms and $P(\sigma(\bar{t})) \rightsquigarrow P(\theta(\bar{t}))$ is derivable with respect to typing rules.

4.1.3 Algorithmic Construction of Synthesis Derivation Trees. The algorithmic construction of synthesis derivation trees is the main focus of TyFlow, whether through the interaction with LMs, or via the translation from type derivation trees. For a synthesis goal of the form $P(\bar{t})$, the task is to synthesize an assignment θ and construct a synthesis derivation tree with the root node labeled $P(\bar{t}) \rightsquigarrow \theta$ by recursively applying synthesis rules as described in Sec. 4.1.1.

This process is detailed in Alg. 1. In this algorithm, only two functions act as black boxes: the selection of synthesis rules and the acquisition of assignments for free variables. Both can be viewed as query problems. Synthesis rule selection is a single-step query that chooses one rule from a finite set of candidates, while assignment acquisition is a multi-step query that constructs a ground term for each free variable using a finite set of constructors and constants.

Consequently, our approach reduces the synthesis problem to finding appropriate solvers for these two specific query problems. This makes our framework more general and modular, as the extracted query resolution problems can be implemented independently. Moreover, the in-time checking mechanisms (unification and constraint verification, etc.) in Alg. 1 significantly restrict the search space compared to naive enumeration-based approaches.

4.2 Query Resolution

Fig. 9 illustrates how query resolution serves as a central mechanism in constructing a synthesis derivation tree. Queries should be resolved to determine which synthesis rule to apply and how to instantiate variables. In the following, we present two complementary approaches for query resolution: employing LMs to make decisions during code generation (Sec. 4.2.1), and translating type derivation trees to prepare supervised training data (Sec. 4.2.2).

4.2.1 Query Resolution using LMs. In the code generation scenario, both query resolution functions are delegated to the LM, which, given the natural language input, acts as a decision-making component for synthesis rule selection and variable assignment.

- **Selection of synthesis rules:** Given the natural language description, the current partially constructed synthesis derivation tree, and the synthesis goal, the LM is expected to choose a specific synthesis rule from the available rule set.
- **Assignment of free variables:** After a synthesis rule is selected and unification is performed, the LM is required to generate a sequence of constructors and constants, constructing ground terms for assigning free variables.

LM prediction is performed in a contextually aware manner, conditioned on multiple sources of information: (1) the natural language describing the desired program behavior; (2) the current synthesis derivation tree, which encodes both the ongoing synthesis process and the type derivation process via the isomorphism between type and synthesis derivation trees; and (3) the synthesis goal $P(\bar{t})$, which includes already synthesized program fragments and the current typing context.

The LM leverages its learned type-reasoning ability to generate the most appropriate synthesis rule and corresponding variable assignments by jointly reasoning about the semantic intentions conveyed by the natural language and the structural requirements encoded in the derivation tree.

4.2.2 Query Resolution through Type Derivation Tree Translation. Beyond the code generation scenario in which natural language serves as input, TyFlow requires curated training data to teach the LM how to interpret and solve synthesis queries. We generate such queries by translating existing type derivation trees of target programs, which involves recording the contextual information and the ground truth decision at each step of synthesis rule application.

The translation is performed by traversing the type derivation tree. When we visit its root node, we begin constructing the synthesis derivation tree from the root node too. For the i -th child labeled $P_i(\bar{x}_i)$ of the current node n_1 in the type derivation tree, upon entering its corresponding subtree, we construct the synthesis derivation tree for the i -th subgoal $P_i(\sigma_{i-1} \dots \sigma_0 \sigma(\bar{x}_i))$ too.

When we return to the node n_1 , the synthesis derivation trees for all subgoals have been constructed, allowing us to build node n_2 in the synthesis derivation tree. At every stage of the synthesis process, there is a node in the type derivation tree corresponding to the current synthesis goal.

- **Selection of synthesis rules:** Assume the node n_1 is labeled with a typing judgment $P(\bar{x}_c)$, annotated with a typing rule R , and the synthesis goal is $P(\bar{x})$. Following the procedure in Sec. 3.3, we translate R into a synthesis rule R' , which is the synthesis rule to be applied.
- **Assignment of free variables:** Assume the typing rule R has the form in Fig. 7, and the synthesis rule R' has the form in Fig. 8. Then $P(\bar{x}_c)$ is the concretization of both the conclusion $P(\bar{x}_0)$ of the typing rule and the synthesis goal $P(\bar{x})$ (can-as-will be proved later). Thus, $\bar{x}_0$ and $\bar{x}$ are unifiable, and the MGU σ makes $\sigma(\bar{x}) = \sigma(\bar{x}_0)$ unifiable with $\bar{x}_c$. Therefore, there exists an assignment θ' such that $\theta' \sigma(\bar{x}_0) = \bar{x}_c$ and θ' covers $\text{FV}(\sigma(\bar{x}_0))$. Since

$FV(\bar{x}_f) \subseteq FV(\bar{x}_0)$, we have $FV(\sigma(\bar{x}_f)) \subseteq FV(\sigma(\bar{x}_0))$, so θ' can also cover $FV(\sigma(\bar{x}_f))$. Let $\sigma_0 = \theta'|_{FV(\sigma(\bar{x}_f))}$ be the assignment for the free variables in the synthesis rule.

Since the translation begins from a valid type derivation tree, it is guaranteed to produce a valid synthesis derivation tree. In particular, the unification, assignment acquisition, and constraint checking steps will always succeed, and the synthesized assignment θ concretizes the original synthesis goal $P(\bar{x})$ to the typing judgment $P(\bar{x}_c)$ labeled at the root of the type derivation tree.

LEMMA 4.2. *Let T_1 be a type derivation tree whose root node is labeled $P(\bar{x}_c)$, annotated with typing rule R , and T_2 be the translated synthesis derivation tree, whose root node is labeled $P(\bar{x}) \rightsquigarrow \theta$. If $\bar{x}$ is unifiable with $\bar{x}_c$, then in the application of the synthesis rule R' that derives $P(\bar{x}) \rightsquigarrow \theta$, (1) the unification step, (2) the acquisition step, and (3) the constraint checking step will succeed, and (4) the synthesized assignment θ concretizes $P(\bar{x})$ to $P(\theta(\bar{x}))$, which is equal to $P(\bar{x}_c)$.*

PROOF. The complete proof can be found in the Appendix A.3. □

With Lemma 4.2, we can conclude that given a type derivation tree T_1 whose root node is labeled *well-typed* (p_c), we can start from the synthesis goal *well-typed* (p), where p is a variable representing the program to be synthesized, and construct a synthesis derivation tree T_2 by translating from T_1 such that the root node of T_2 is labeled *well-typed* (p) $\rightsquigarrow \theta$, where $\theta(p) = p_c$.

4.3 Isomorphism between Type Derivation Trees and Synthesis Derivation Trees

Since every synthesis rule is translated from a typing rule, and every synthesis subgoal in the synthesis rule corresponds to a premise in the typing rule, we can establish an isomorphism between synthesis derivation trees and type derivation trees.

Isomorphism of Derivation Trees. A type derivation tree $T_1 = (V_1, r_1)$ (with node set V_1 and root r_1) and a synthesis derivation tree $T_2 = (V_2, r_2)$ are *isomorphic* if there exists a bijection $\varphi : V_1 \rightarrow V_2$ satisfying the following conditions:

- (1) Root Correspondence: $\varphi(r_1) = r_2$
- (2) Child Order Preservation: For $v \in V_1$, if $\text{children}_{T_1}(v) = [v_1, \dots, v_k]$, where $\text{children}_T(v)$ denotes the ordered list of v 's children in tree T , then $\text{children}_{T_2}(\varphi(v)) = [\varphi(v_1), \dots, \varphi(v_k)]$
- (3) Label/Annotation Correspondence: For every $v \in V_1$ labeled with typing judgment $P(\bar{t}_c)$ and a typing rule, $\varphi(v)$ is labeled with synthesis judgment $P(\bar{t}) \rightsquigarrow \theta$, annotated with the corresponding synthesis rule and assignment, where $\theta(\bar{t}) = \bar{t}_c$

We write $T_1 \cong T_2$ to denote that T_1 and T_2 are isomorphic, representing the same logical structure but in different forms. The translation procedure described in Sec. 4.2.2 provides a systematic way to construct an isomorphic synthesis derivation tree from any given type derivation tree. During this process, for each node n_1 in the type derivation tree, we define the correspondence $\varphi(n_1) = n_2$, where n_2 is the node constructed by translating n_1 . It is straightforward to verify that this bijection preserves the root nodes and the order of children, and, by Lemma 4.2, also preserves the correspondence of labels and annotations. Thus, $T_1 \cong T_2$, as captured by the following lemma.

LEMMA 4.3. *For any type derivation tree T_1 , there exists a synthesis derivation tree T_2 , $T_1 \cong T_2$.*

Conversely, we can construct a type derivation tree T_1 from a given synthesis derivation tree T_2 , also preserving isomorphism. The construction is straightforward: we traverse T_2 , and for each node labeled with the synthesis goal

$P(\bar{t}) \rightsquigarrow \theta$, we create a corresponding node labeled with the typing judgment $P(\theta(\bar{t}))$ and annotate it with the appropriate typing rule. Similar to Lemma 4.2, we can inductively show that the resulting T_1 is a type derivation tree, and that the bijection between T_1 and T_2 ensures the isomorphism $T_1 \cong T_2$.

LEMMA 4.4. *For any synthesis derivation tree T_2 , there exists a type derivation tree T_1 , $T_1 \cong T_2$.*

PROOF. The complete proof can be found in the Appendix A.4. □

4.4 Soundness and Completeness

We have established an isomorphism between type derivation trees (type correctness proofs) and synthesis derivation trees (existential type correctness proofs). Under the isomorphism, TyFlow ensures that well-typed programs can always be *extracted* from synthesis derivation trees, rather than relying on direct program synthesis without reference to their type correctness proofs.

The notions of “extract” and “well-typed” can be formally defined in the context of derivation trees. Given a synthesis derivation tree with root node labeled *well-typed* $(p) \rightsquigarrow \theta$, the extracted program is $\theta(p)$. ~~And a~~ A program p_c is said to be well-typed if there exists a type derivation tree (type correctness proof) whose root node is labeled *well-typed* (p_c) .

THEOREM 4.5 (SOUNDNESS). *Any program extracted from a synthesis derivation tree T is well-typed.*

PROOF. Suppose the root node of T is labeled *well-typed* $(p) \rightsquigarrow \theta$. By Lemma 4.4, there is a type derivation tree T' such that $T \cong T'$. The root node of T' is labeled *well-typed* (p_c) , where $p_c = \theta(p)$ is the program extracted from T . By the definition of well-typed programs, p_c is well-typed. □

THEOREM 4.6 (COMPLETENESS). *For any well-typed program p_c , there exists a synthesis derivation tree such that the program extracted from it is p_c .*

PROOF. There exists a type derivation tree T_1 whose root node is labeled *well-typed* (p_c) . By Lemma 4.3, there exists a synthesis derivation tree T_2 such that $T_1 \cong T_2$. The root node of T_2 is labeled *well-typed* $(p) \rightsquigarrow \theta$, where $\theta(p) = p_c$. Thus, the program extracted from T_2 is p_c . □

COROLLARY 4.7. *The synthesis derivation tree T guaranteed by the completeness theorem is precisely the tree obtained through the translation process described in Sec. 4.2.2.*

The soundness and completeness theorems ~~ensure~~ establish the correspondence between well-typed programs and synthesis derivation trees. ~~This correspondence works~~ in both directions: every well-typed program can be represented as a synthesis derivation tree (completeness), and every synthesis derivation tree yields a well-typed program (soundness). Together, these results confirm that constructing a synthesis derivation tree is equivalent to constructing an existential type correctness proof: the isomorphism with type derivation trees provides the proof structure, while the synthesized assignment provides the witness (the program) that makes the existential statement $\exists p. \text{well-typed}(p)$ hold.

The two theorems characterize the search space of TyFlow: they guarantee that it is neither weakened nor strengthened, but say nothing about how the search proceeds inside that space. That is what the decision-sequence representation, model training, and the pruning techniques are for: they make the search more effective without moving its boundary.

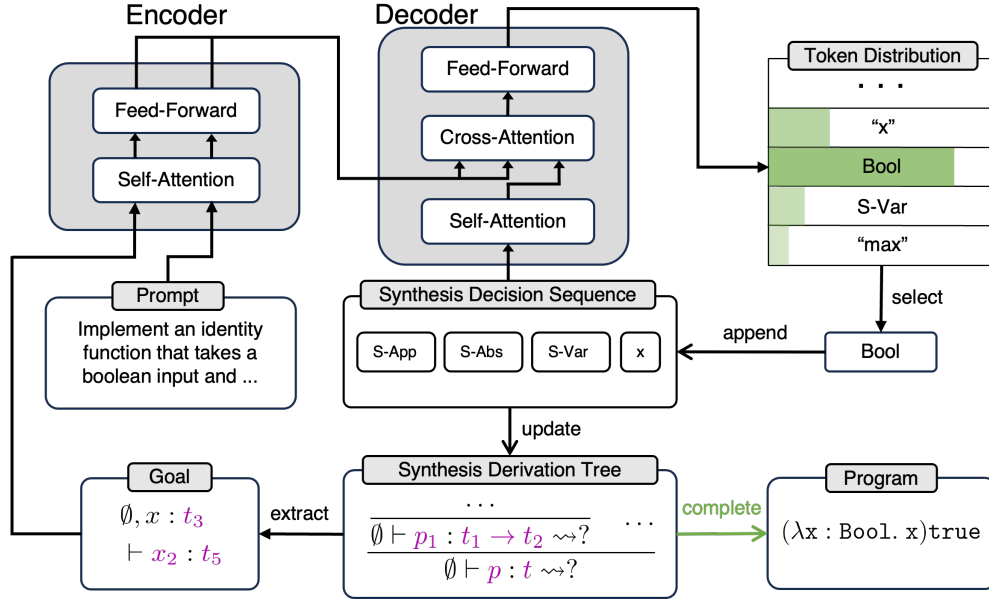


Fig. 10. Model Architecture for Constructing Synthesis Derivation Trees

5 Language Model Design

5.1 Model Architecture

To better adapt to our proof-guided synthesis system and maximize token utilization efficiency, we design a novel encoder-decoder-based model architecture, illustrated in Fig. 10.

Unlike standard encoder-decoder models that encode only a static input, our architecture features a *dual-encoding* design that separately processes two distinct input streams: (1) the fixed natural language specification, and (2) the dynamically evolving synthesis goal at each step.

Static Encoding. The natural language specification remains unchanged throughout the synthesis process. We encode it once at the beginning to produce a fixed sequence of hidden states that stays constant across all synthesis steps.

Dynamic Encoding. The synthesis goal, which provides the *dynamic typing context*, evolves at each step as the synthesis derivation tree grows. We re-encode the current synthesis goal at each step to capture the updated typing context. The encoder produces a separate sequence of hidden states for this dynamic input.

Cross-Attention Fusion. The decoder accesses both encodings through cross-attention mechanisms. We concatenate the static encoding H_s and dynamic encoding H_d along the sequence dimension to form a unified key-value sequence $H = [H_s; H_d]$. The decoder’s cross-attention layers then attend to this concatenated sequence, computing attention over both the task specification and the current typing context simultaneously. This design allows the model to jointly reason about both information sources when predicting the next synthesis decision.

As described in [Sec. 4.2.1](#), the query resolution task is delegated to the LM, which takes the natural language prompt, the synthesis decisions generated so far, and the current synthesis goal as input and outputs a token representing the synthesis rule or variable assignments.

Meanwhile, TyFlow maintains a synthesis decision sequence where each newly generated token is appended, representing the next synthesis decision. This sequence incrementally constructs an existential type correctness proof. Due to its autoregressive nature, the growing sequence is fed to the transformer decoder, allowing the LM to leverage the full history of synthesis decisions when predicting the next token.

Model Inference Process. At each synthesis step:

- (1) The current synthesis goal is extracted from the (incomplete) synthesis derivation tree.
- (2) The encoder encodes the synthesis goal to provide *dynamic typing context* to the LM.
- (3) The decoder processes both encodings from the encoder, along with the previous synthesis decision sequence, generating the next token representing a synthesis decision.
- (4) This new token is appended to the synthesis decision sequence and used to incrementally update the synthesis derivation tree as detailed in [Alg. 1](#).
- (5) The updated synthesis derivation tree yields either a new synthesis goal for the next iteration or, upon completion, a well-typed program together with its type correctness proof.

Efficient Token Reuse through Incremental Construction. According to [Alg. 1](#), the synthesis derivation tree construction follows a serialized order, ensuring that each step only appends new decisions without modifying the existing sequence. This property aligns naturally with the autoregressive generation paradigm of transformer decoders, where each new token is predicted based on all previously generated tokens. As a result, we can reuse the transformer’s key-value (KV) cache [\[27\]](#): the intermediate attention states computed for earlier tokens in the decision sequence are cached and reused when generating subsequent tokens. This eliminates the need to recompute attention over the entire decision sequence at each decision-making step, significantly reducing the computational overhead from quadratic to linear in sequence length.

Model Configuration. Our dual-encoding architecture is built upon standard Transformer encoder-decoder models, where both the encoder and decoder follow the standard Transformer architecture [\[34\]](#). We evaluate on two pre-trained models: **CodeT5-220M** [\[35\]](#), with a 12-layer encoder and a 12-layer decoder, 12 attention heads, a hidden dimension of 768, and approximately 220 million parameters; and **T5Gemma2-2B** [\[39\]](#), with a 26-layer encoder and a 26-layer decoder, 8 attention heads, a hidden dimension of 2304, and approximately 2 billion parameters. For both models, the static encoder and dynamic encoder share the same weights but process different input streams.

5.2 Term Encoding

Terms, as defined in [Sec. 3.2.1](#), are inductively constructed from constants and constructors. Following the approach of GrammarT5 [\[42\]](#), we serialize the abstract syntax tree (AST) of a term using a pre-order traversal, outputting each constructor or constant as a token in sequence. When a term contains unknown variables (not yet instantiated during synthesis), we represent them using a unified unknown token $\langle ? \rangle$.

For example, consider the STLC program `abs("x", bool, var("x"))`, which represents the identity function $\lambda x:\text{bool}. x$. Its AST has the constructor `abs` at the root, with three children: the constant `"x"` (the variable name), the constant `bool` (the type annotation), and the subtree `var("x")`. The pre-order traversal yields the token sequence illustrated in [Fig. 11](#).

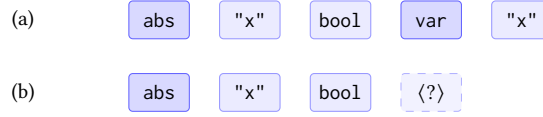


Fig. 11. Term encoding as token sequences: `constructors` and `constants`. (a) Full term `abs("x", bool, var("x"))`. (b) Partial term `abs("x", bool, e)` with unknown variable.

This encoding preserves the structural information of the term and can be uniquely decoded back to the original AST. The decoding is possible because each constructor has a fixed arity and known parameter types (as specified in the term definitions in [Sec. 3.2.1](#)). When the decoder encounters a constructor token, it knows exactly how many subsequent tokens to consume and what types they should have, enabling unambiguous reconstruction of the tree structure. If the number of arguments or their types do not match the constructor's specification, the decoding fails and reports an error. During synthesis, the LM generates tokens following this same pre-order traversal order to construct terms for variable assignments. As each token is generated, the synthesis system incrementally decodes the sequence to reconstruct the corresponding AST, enabling immediate validation and integration into the synthesis derivation tree.

5.3 Advanced Techniques for LM-Based Program Synthesis

The program synthesis algorithm in [Alg. 1](#) involves sequential actions at each node: selecting synthesis rules, acquiring variable assignments, and applying rules to update the synthesis derivation tree. To ensure the efficiency and correctness of this process, we implement pruning strategies that naturally correspond to the potential failures in synthesis rule application.

At the syntactic level, we apply *syntactic pruning*, which ensures the structural validity of synthesis derivation trees and enforces the well-formedness of generated sequences. *Syntactic pruning* is triggered instantly ~~when~~after a token is generated in the following cases: (1) the synthesis goal expects a synthesis rule to apply but the token stands for a term, or vice versa; (2) the predicate in the synthesis rule's conclusion differs from that of the synthesis goal; or (3) during term generation, the token corresponds to constructors or constants (see [Sec. 3.2.1](#)) that mismatch the expected term type.

Example 5.1. Consider generating the term `abs("x", bool, var("x"))` using the term encoding from [Sec. 5.2](#). Suppose the current token sequence is `[abs, "x"]`, meaning the LM has generated the constructor `abs` and its first argument `"x"` (the variable name). According to the constructor specification `abs : (String, Type, Prog)`, the next token should be of type `Type`. If the LM generates `var` (a `Prog` constructor) instead of a valid type like `bool`, syntactic pruning immediately rejects this token because the expected type `Type` does not match the actual type `Prog`.

At the type level, we apply *type pruning*, which focuses on type-level correctness and relies on premise checking during the application of synthesis rules. If the unification step or constraint checking step fails, the corresponding search is immediately terminated. This prevents the exploration of synthesis paths that cannot lead to valid type correctness proofs.

Example 5.2. Consider applying the T-VAR rule, which has a constraint $x : t \in \Gamma$ that checks whether the variable x with type t exists in the context Γ . Suppose the current synthesis goal is $\Gamma \vdash e : \text{int}$ where $\Gamma = \{f : \text{bool}, g : \text{int}\}$. If the LM selects f to instantiate e , the constraint checking step verifies $f : \text{int} \in \Gamma$. Since f has type `bool` in Γ , not `int`, the

constraint fails and type pruning terminates this branch. The correct choice would be g , which has the matching type `int`.

Syntactic pruning, *type pruning*, and the previously mentioned *dynamic typing context* constitute a layered enhancement framework: *syntactic pruning* serves as a prerequisite for *type pruning*, since a structurally invalid synthesis derivation cannot be applied to perform type-level checking. Furthermore, without the successful application of synthesis rules, the intended synthesis goal cannot be extracted from the synthesis derivation tree. Should all three techniques be omitted, the model reduces to a standard encoder-decoder architecture that directly maps natural language inputs to sequences of synthesis decisions, lacking any mechanism to enforce structural or type-level correctness.

Beam search [8] enables efficient exploration of multiple synthesis paths by maintaining the top- k candidate sequences based on cumulative likelihood scores. At each step, rather than greedily selecting only the highest-probability token, beam search expands all k beams and retains the top- k sequences across all candidates. When a branch is pruned due to syntactic or type errors, we terminate its sequence and discard its KV cache, while the remaining branches continue independently. This prevents invalid branches from wasting beam slots, ensuring all k slots are reserved for branches that may lead to valid programs.

6 Evaluation

We design our evaluation to answer the following research questions.

- **RQ1:** How does TyFlow compare with ~~the existing code generation methods~~ its base models across model scales and benchmarks?
- **RQ2:** How does each component of TyFlow contribute to the overall performance, how does the pruning behave, and what runtime cost does it introduce?
- **RQ3:** How does ~~the correctness of~~ TyFlow compare with other reliability-oriented generation approaches, including rejection sampling, the previous method for ensuring the correctness of code generation token-level constrained decoding, completion-engine repair, and iterative compiler-feedback repair?
- **RQ4:** How does TyFlow compare with variants that generate the type derivation tree and the program separately instead of simultaneously?

6.1 Experimental Setup

6.1.1 Benchmark. To evaluate the generality of TyFlow, our evaluation considers two languages with very different characteristics, SuFu [15] and Java.

- **SuFu** is an ML-style functional language designed for automatic program optimization. This language uses a customized type system for extracting operations for optimization on data structures. This language has limited code resources, making it representative for evaluating TyFlow under low-resource conditions. We collect SuFu programs from its GitHub repository, which comprises 290 programs with test cases. For each program, we manually supply a natural language description, along with the relevant inductive definitions and library functions, thus collecting a dataset of code generation tasks. This dataset is then split randomly into a training set of 80% tasks and a test set of 20% tasks.
- **Java** is a popular imperative language with a sophisticated type system. ~~In a balance between the~~ To balance implementation cost and ~~the~~ expressiveness, we implement a subset of Java that preserves its core syntax while omitting advanced features like lambda expressions.

We consider the MBJP dataset [1] in our evaluation, which is the Java version of the popular MBPP dataset [5]. Our Java subset covers ~~608 tasks from MBJP, and we split these tasks~~ the MBJP tasks admitted by our toolchain, ~~which we split~~ randomly into a training set of ~~90%-608~~ tasks and a ~~held-out~~ test set of ~~10%-67~~ tasks.

- HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java [5], and TransCoder-GFG collects the Java programs released with TransCoder [18]; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

More information about SuFu, Java, and the ~~datasets~~ dataset construction can be found in Appendix B.

6.1.2 Models. We implement TyFlow based on CodeT5 [35], a family of encoder-decoder models for code understanding and generation. We adopt the 220M variant of CodeT5 as our base model for subsequent training and refer to our model as TyFlow-220M. We also evaluate TyFlow on T5Gemma2 [39], a more recent encoder-decoder model family built upon Gemma 3. We adopt the 2B variant of T5Gemma2 as our base model and refer to this model as TyFlow-2B.

6.1.3 Evaluation Metrics. To evaluate the quality of code generation, we adopt the metrics below, which respectively measure the correctness, effectiveness, and type correctness:

- **Pass@k (k=1,10) [5]:** These metrics quantify the proportion of problems where at least one of the top-k generated candidates passes all reference test cases. Each generated candidate is executed against the test cases provided in the benchmark, and a candidate is considered correct only if it produces the expected output for all test cases.
- **First Success Position (FSP):** This metric is the mean rank of the first functionally correct candidate among the generated ones across the test set. It measures how early in the candidate list a correct solution appears, assessing the efficiency of output ranking; a problem with no correct candidate contributes the candidate-list length (10).
- **Compilation Error Rate (CER):** This metric quantifies the proportion of generated candidates that fail to compile. Since TyFlow generates programs with type correctness guarantees, CER reflects the type system’s effectiveness in catching compile-time errors.

6.1.4 Implementation Details. We fine-tuned both TyFlow-220M (based on CodeT5-220M) and TyFlow-2B (based on T5Gemma2-2B) following a standard supervised fine-tuning procedure with the same set of hyperparameters to ensure a fair comparison. Each model was fine-tuned on the training set until convergence, ~~with multiple checkpoints saved throughout the process. The best-performing checkpoint is used for evaluation.~~. Since TyFlow introduces a new representation that differs from standard code tokens, ~~we and thus faces a cold-start problem, we first~~ perform an initial ~~fine-tuning-training~~ phase using a supervised fine-tuning dataset from OpenCoder [14] to ~~help the model adapt~~ adapt the model to this representation. At the same scale and on the same benchmark, each baseline-TyFlow pair is fine-tuned on the same training set. The exception is the two additional Java benchmarks, HumanEval-Java and TransCoder-GFG. There, TyFlow-2B builds on its MBJP fine-tuning, which avoids the cold-start problem, and is further trained on the training splits of the two benchmarks. The baselines, which generate ordinary code tokens and face no cold-start problem, are instead trained from the original T5Gemma2-2B, with the MBJP training tasks added to their own training sets for joint training.

6.2 Experimental Results

6.2.1 *RQ1: Performance of TyFlow.* Tab. 2 summarizes the overall performance of TyFlow, with models grouped by scale. The top four models are fine-tuned and evaluated on the SuFu dataset, while the bottom four models are on the MBJP dataset. For each language model scale and benchmark, we compare the baseline models fine-tuned baseline (CodeT5-220M and/or T5Gemma2-2B) with their corresponding variants our TyFlow model on the metrics above; the 2B group additionally covers the two Java benchmarks, HumanEval-Java and TransCoder-GFG.

Table 2. Comparison of Accuracy and Compilation Error Rate between CodeT5-Base Models and TyFlow on SuFu at the 220M and Java Datasets 2B Scales

Scale	Benchmark	Model	pass@1 (%) [↑]	pass@10 (%) [↑]	FSP [↓]	CER (%) [↓]
220M	SuFu	CodeT5-220M	24.14	32.76	7.03	83.10
		TyFlow-220M	37.93	53.45	5.03	0.00
	MBJP	CodeT5-220M	10.45	20.90	8.19	35.52
		TyFlow-220M	11.94	28.36	7.94	1.53
2B	SuFu	T5Gemma2-2B	25.86	37.93	6.66	71.21
		TyFlow-2B	36.21	48.28	5.53	0.00
	MBJP	T5Gemma2-2B	13.43	32.84	7.46	29.55
		TyFlow-2B	25.37	43.28	6.36	0.45
	HumanEval	T5Gemma2-2B	31.25	37.50	6.50	24.38
		TyFlow-2B	50.00	56.25	4.75	1.30
	GFG	T5Gemma2-2B	19.42	33.01	7.10	13.69
		TyFlow-2B	30.10	46.60	5.81	2.75

[↑] Higher is better; [↓] Lower is better; **bold** indicates best value per column; pass@k measures the proportion of problems solved by at least one of the top-k candidates (verified by test cases); FSP is the average rank of the first correct candidate; CER is the compilation error rate. See metrics in Sec. 6.1.3 for details.

TyFlow consistently improves code generation accuracy across both base models. On SuFu, 220M improves at both scales and on every benchmark: pass@10 from 32.76% (CodeT5-220M) to 46.55%, and 2B improves it from 37.93% (T5Gemma2-2B) to 48.28% (SuFu) and from 10.45% to 11.94% (MBJP) at 220M, and from 25.86% to 36.21% (SuFu), 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 from 20.90% to 28.36%, improves by 7.46–20.69 points on the same six benchmarks. The largest gains are on SuFu and 2B improves it from 35.82% to 40.30% HumanEval-Java, where the type constraints remove most uncompileable candidates while the base models often still produce them. At 220M, the Java baseline has a lower CER than the SuFu baseline (35.52% versus 83.10%), leaving fewer compilation errors for type constraints to address. On MBJP, TyFlow raises pass@10 from 20.90% to 28.36% and reduces CER to 1.53%; the smaller pass@1 gain indicates that finding a correct candidate does not always make it the top-ranked output. The failure analysis in Appendix C identifies ranking errors and compilable but incorrect programs among the remaining failures. TyFlow also drops CER to 0.00% on SuFu and 0.45% / 1.30% / 2.75% on the three Java benchmarks; the small residual Java CER comes from static-analysis errors the type system does not capture. The improvements are particularly pronounced on SuFu.

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, indicating that enhances the model’s ability to reason about type constraints on languages with complex type systems

and limited training data. Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task: the zero-shot condition uses only the task prompt, and the few-shot condition prepends three solved examples for the Java benchmarks and six examples for SuFu.

In terms of the position of the first successful prediction (FSP), On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8 of 58, whereas TyFlow consistently reduces FSP compared to the corresponding base models. On SuFu, -220M reduces FSP from 7.03 to 5.48, and -2B reduces it from 6.69 to 5.03. -2B solves 21; the low-resource language is where the type-guided representation delivers its advantage. On Java, given the task prompt without examples, the three larger models solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks at pass@1. TyFlow -220M reduces FSP from 8.19 to 7.94. -2B solves 17/67, 8/16, and 31/103, respectively. Moreover, few-shot prompting on Java brings little or no gain over zero-shot for these models, which suggests that they have seen this language extensively and may even have been trained on these benchmarks directly; we therefore flag a possible data-leakage risk for the Java comparison. In any case, relative to its fine-tuned baseline T5Gemma2-2B, TyFlow-2B reduces it from 6.99 to 6.76, delivering a consistent improvement on all four benchmarks.

Most notably,

Table 3. Pass@1 Comparison with Larger Decoder-Only Models

Model	MBJP (67)		HumanEval-Java (16)		TransCoder-GFG (103)		SuFu (58)	
	0-shot	few-shot	0-shot	few-shot	0-shot	few-shot	0-shot	few-shot
MiMo-7B	34	33	3	10	38	43	0	6
Qwen3-14B	42	41	14	11	50	49	0	8
Qwen3-30B-A3B	42	45	13	14	52	52	0	6
T5Gemma2-2B		9		5		20		15
TyFlow-2B		17		8		31		21

Cell entries are solved tasks at pass@1 for all models; the 0-shot condition uses the same prompt as the two fine-tuned models below.

Overall, achieves clear performance gains and significant improvements in code correctness and reliability for both programming languages, fulfilling our design objectives.

6.2.2 RQ2: Contributions, Pruning Behavior, and Runtime Cost of Individual Components. To answer RQ2, we conduct an ablation study on the SuFu benchmark and the TyFlow-220M model to quantify the contributions of each component in TyFlow. Starting from the base case where TyFlow synthesizes decision sequences with no checks, system components (syntactic pruning, type pruning, and dynamic typing context, as introduced in Sec. 5) are incrementally added, and the impact on performance metrics is evaluated at each stage. Tab. 4 summarizes the results.

Starting from the base setting, each subsequent enhancement contributes meaningfully to the overall improvement in code-generation quality and reliability.

Syntactic pruning improves

Functional contribution of each component. The four metric columns of Tab. 4 show which component contributes which improvement. Syntactic pruning alone gives a moderate pass@1 by +3.45% and reduces CER by 2.13%, demonstrating that basic structural constraints help filter trivially invalid outputs. Type pruning delivers transformative results by

Table 4. Sequential Ablation Study: Performance with Incremental Component Additions

Components	pass@1 (%) [†]		pass@10 (%) [†]		FSP [↓]		CER (%) [↓]		Time (s) [§]	ms/step [§]
	Score	Δ	Score	Δ	Score	Δ	Score	Δ		
Base (no checks)	24.14	0.00	36.21	0.00	6.78	0.00	74.26	0.00	5.70	31.1
+ Syntactic Pruning	27.59	3.45	36.21	0.00	6.71	-0.07	72.13	-2.13	5.34	32.4
+ Type Pruning	37.93	+13.79	43.10	6.89	5.79	-0.99	0.00	-74.26	11.07	43.7
+ Dynamic Typing Context	37.93	+13.79	53.45	+ 17.24	5.03	-1.75	0.00	-74.26	15.62	59.6

[†]*ValScore*: absolute value for each setting; Δ: improvement relative to the base setting.

[§]*Time*: average per-task wall time; *ms/step*: average wall time per decoding step.

eliminating all compilation errors (CER drops to 0.00%) while substantially boosting gain (+3.45); the turning point is type pruning, which lifts pass@1 by +13.79 and eliminates compilation errors outright (CER 74.26 → 0.00). Dynamic typing context leaves pass@1 to 37.93% (+13.79%) and CER unchanged but improves pass@10 to 43.10% (+6.89%), highlighting type correctness as critical for synthesis. Dynamic typing context further enhances performance, pushing pass@10 to 53.45% (+17.24 over the base) and FSP: it ranks correct candidates earlier, which demonstrates the **Context Locality** property of Sec. 1.

How much the constraints prune. To measure the pruning strength, we instrument the constrained decoder on the SuFu tasks (beam 10 to 46.55% (+10.34%) and reducing FSP to 5.48 (-1.30), showing that richer typing context enables generating more correct solutions among higher-ranked outputs.

The results validate our motivation: the type-based mechanisms are essential for ensuring type correctness. In particular, 330-step budget²): syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2%. The lower rate is expected: type checking is more expensive, so it is applied only to the effect of high-probability candidates about to enter the beam. Such candidates are rarely ill-typed, yet pruning them matters, for they would otherwise expand within the beam and crowd out well-typed derivations. Type checking during search is provisional (only completed AST fields are checked); the whole program is checked once when the derivation finishes, and this final check rejects 21.2% of the completed candidates.

Whether the constraints over-prune. Completeness guarantees that a well-typed program has a derivation, but it does not guarantee that the model ranks a correct prefix within a fixed beam or provide a bound on the required beam size. A finite beam can discard all correct prefixes, and the retained hypotheses may later have no valid continuation, leaving the active beam empty. Training on correct decision sequences is intended to rank correct prefixes highly, but cannot rule out this case. We measure the effective branching factor as the number of active hypotheses present at each decoding step after pruning. With beam 10, this number averages 7.97. It is lower than 10 because completed programs are moved to the output set, while end-of-sequence and rejected candidates do not fill active slots; the top-20 candidate list can therefore be exhausted before all slots are filled. In our experiments, however, the active beam never became empty.

Runtime cost. Each component increases the per-step cost (31.1 → 32.4 → 43.7 → 59.6 ms). Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context demonstrates the importance of **Context Locality** in - by providing necessary type information within a compact local context, this technique significantly improves the

²The longest decision sequence in the benchmark has 330 steps, so the budget suffices to derive every reference program.

~~effectiveness-of-generation~~cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal. The per-task totals move by -6.38% / $+94.00\%$ / $+173.88\%$ ($2.74\times$ base). Syntactic pruning even lowers the per-task total: with little per-step overhead, it prunes ill-formed decisions early, so beams stay on well-formed derivations that finish sooner (164.6 versus 183.2 mean steps per task), and the 10% fewer steps outweigh the per-step increase.

6.2.3 RQ3: Comparison with Rejection-Sampling*Reliability-Oriented Generation Methods*. To answer RQ3, we compare with two baselines: the vanilla CodeT5-220M (without any post-processing) and CodeT5-220M with rejection sampling. We compare TyFlow with methods that pursue correct code by constraining or repairing the output of a trained model. On MBJP, four such methods are applied on top of the same frozen T5Gemma2-2B checkpoint selected in RQ1, adding their own pruning or repair mechanisms on top: *rejection sampling*, the previous state-of-the-art for achieving guarantees in code generation. In our case, rejection sampling queries the LM repeatedly and discards those candidate programs that cannot be compiled until finding sufficient solutions or reaching a resource limit. Notably, rejection sampling is functionally equivalent to constrained decoding, as both methods effectively re-normalize the model’s output distribution over the subspace of valid, compilable programs. By including rejection sampling, we establish a strong baseline for the performance of the model under strict correctness constraints. *SynCode* [33], *Repilot* (Copiloting the Copilots) [36], and *iterative compiler repair*. All methods are evaluated on the same 67 tasks with 10 candidates each. Because rejection sampling and repair need diverse draws rather than beam output, the ordinary control uses the temperature-sampling decoding protocol (temperature 0.8, top- p 0.95), so its scores (14.93/23.88) differ from the beam-10 row of Tab. 2 (13.43/32.84); the TyFlow-2B row is identical in both tables.

Rejection sampling repeatedly draws candidates and keeps those that pass Java compilation, for up to ten ten-draw rounds (100 candidates) per task. For SynCode, we reuse its released grammar-masking code and Java grammar with T5Gemma2 on MBJP. At each step it masks the tokens that would violate a given grammar, so every output is syntactically well-formed. For Repilot, we reuse its modified Eclipse JDT completion engine and adapt it to T5Gemma2 on MBJP. During generation, the partial Java program is maintained as a JDT document, and completion queries guide token pruning. Following the original policy, punctuation and keywords bypass these queries. Iterative compiler repair feeds javac diagnostics back to the model and regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics. We cap repair at two rounds per candidate to bound the additional generation cost and stop earlier if the candidate compiles.

Table 5. Comparison ~~of Vanilla CodeT5, Rejection Sampling, and of~~ TyFlow ~~with~~ *Reliability-Oriented Generation Methods on Java*

Method	pass@1 (%) [↑]	pass@10 (%) [↑]	CER (%) [↓]
T5Gemma2-2B	14.93	23.88	26.12
+ Rejection Sampling	16.42	28.36	0.00
+ SynCode (adapt., compile-safe)	14.93	23.88	21.34
+ Repilot/JDT	14.93	23.88	25.97
+ Iterative compiler repair	14.93	23.88	17.31
TyFlow-2B	25.37	43.28	0.45

^{↑/↓} Higher/lower is better.

On both datasets, rejection sampling can provide only minor improvements over the baseline, much less than the improvements achieved by None of the four external baselines significantly improves functional correctness over the ordinary control: rejection sampling adds three solved tasks at pass@10 (19 vs. 16 of 67) and one at pass@1 (11 vs. 10; McNemar $p = 1.00$ and $p = 0.25$); SynCode, Repilot, and iterative repair leave the solved sets exactly unchanged (10/67 and 16/67); Following the original Repilot implementation, punctuation and keyword tokens bypass completion queries; in this configuration, CER decreases from 26.12% to 25.97%. Iterative repair reduces compilation failures from 175 to 116 of 670 candidates, but none of the 59 candidates repaired to compile passes the functional tests. Compiler diagnostics help resolve compilation errors, but do not establish whether a program implements the intended behavior, explaining why improved compilability does not translate into additional solved tasks here. TyFlow. This demonstrates that rejection sampling, as a post-hoc filtering technique, fails to enhance the LM’s ability to generate correct code and is thus limited by the candidate quality produced by the base LM. Similarly, constrained decoding, which also relies on filtering during generation, would face comparable limitations as it cannot fundamentally improve over the base LM, either.

In contrast, consistently achieves higher 2B solves 17/67 at pass@1 and 29/67 at pass@10, reflecting its ability to generate more correct and top-ranked predictions. Unlike filtering-based methods, explicitly incorporates typing rules and typing context during training, enabling the model to learn and utilize type constraints. These results highlight that embedding type information into LMs is substantially more effective than post-hoc pruning approaches, demonstrating the critical importance of with 3 of 670 candidates failing to compile, the best of all compared methods. The evaluated external mechanisms constrain or revise generated programs without updating the underlying model parameters, whereas TyFlow explicitly encodes the typing rules into the model and helps it better learn the program distribution, supporting the value of the **Type Explicitness** in-property in Sec. 1.

6.2.4 RQ4: Comparison with Separated Type-Code Generation. To answer RQ4, we We compare TyFlow with approaches that separate type reasoning and code generation. We evaluate three variants: (1) Type-First: Both variants fine-tune CodeT5-220M is fine-tuned to generate a sequence to emit a single sequence that concatenates the two parts, differing only in their order: Type-First generates the sequence of typing rules (forming the type derivation tree) as explicit type reasoning, then generate code based on the reasoning, (2) Code-First: CodeT5-220M is finetuned to first generate code, then output a sequence of first and then the code on its basis, whereas Code-First generates the code first and then the typing rules as its type correctness proof, and (3) 220M: our synthesis decision sequence representation reasoning. All settings are evaluated on the same SuFu and MJP test sets with 10 candidates each.

compares separated reasoning approaches against on both SuFu and Java benchmarks. Both sequential variants achieve Adding separated type reasoning improves over the ordinary trained baseline on both languages (e.g., Type-First 31.03 vs. 24.14 pass@1 on SuFu), and unifying the two into one process improves further: both separated variants trail TyFlow on pass10 of 36–40% on SuFu and 22–27% on Java, while achieves 46.55% and 28.36% respectively. Moreover, they still suffer from high compilation error rates (57–64% on SuFu and 31–38% on Java CER (on Java pass@1, Type-First merely ties TyFlow). Because neither variant constrains decoding, their CER remains high (56.72–64.31 on SuFu), whereas eliminates compilation errors on SuFu and reduces them to 3.52% on Java.

Besides the correctness advantages, also outperforms both baselines in token usage. In comparison, reduces token consumption by 45% on SuFu (410 vs. 752 tokens) and 48% on Java (TyFlow outputs only type-valid programs. In terms of token length, fully unifying type reasoning and code is far more economical than outputting them separately: the

Table 6. Comparison: Separated vs. Integrated Type-Code Generation

Language	Model	pass@1 (%) [↑]	pass@10 (%) [↑]	CER (%) [↓]	FSP [↓]	Avg Tokens [↓]
SuFu	CodeT5-220M	24.14	32.76	83.10	7.03	405
	CodeT5-220M + Type-First	31.03	36.21	64.31	6.57	752
	CodeT5-220M + Code-First	29.31	39.66	56.72	6.28	752
	TyFlow-220M	37.93	53.45	0.00	5.03	410
Java	CodeT5-220M	10.45	20.90	35.52	8.19	142
	CodeT5-220M + Type-First	11.94	26.87	37.61	7.36	247
	CodeT5-220M + Code-First	8.96	22.39	31.49	8.21	247
	TyFlow-220M	11.94	28.36	1.53	7.94	129

^{↑/↓} Higher/lower is better.

Avg Tokens: length of the generated sequence in each model's own vocabulary, excluding comments.

unified decision sequence (410 on SuFu, 129 vs. on Java) saves 342 and 118 tokens respectively against appending the typing rules to plain code tokens (752 and 247 tokens) compared to the baselines.

These results validate ~~s~~). It is even comparable to, or slightly shorter than, the plain code tokens alone (405 on SuFu, 142 on Java): a type derivation rule absorbs the redundant syntactic elements of the code (such as equals signs and semicolons) into a single rule token, so our representation adds little and sometimes removes tokens. These results support the **Derivation Vicinality** paradigm :by unifying type reasoning and code generation into a single synthesis process, of TyFlow achieves both higher accuracy and improved efficiency, that interleaving each program fragment with its type derivation is more accurate and more efficient than generating them separately.

6.3 Limitations and Future Work

Dependence on the encoder-decoder architecture. The current architecture is designed for encoder-decoder models: at each synthesis step, an encoder processes the evolving synthesis goal to provide the dynamic typing context. The decision-sequence representation and synthesis rules can be used unchanged with a decoder-only model. One adaptation would append the current typing context to the decision tokens generated so far as temporary input for predicting the next decision, then replace it with the updated context before the following prediction. This requires recomputing the context's key-value entries at each step and adapting cache management to the changing context. Implementing this decoding procedure and evaluating its runtime are directions for future work.

Type-system expressiveness. Our evaluation covers a Java subset and SuFu, while the CHC-based construction provides a path to richer type systems through rules in the supported form and suitable unification and constraint-checking procedures. For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost. Rules requiring higher-order unification would need a suitable solver for a supported decidable fragment (Sec. 3). This extension path also applies to statically checkable properties of dynamically typed languages such as Python: we can select a statically checkable subset and model its typing rules. Automatically recovering derivations under these rules would provide training decision sequences, while the corresponding synthesis rules would ensure that generated programs satisfy the modeled static checks, rather than guarantee all runtime behavior. Implementing and evaluating these extensions remains future work.

7 Related Work

7.1 Rejecting Invalid Generations

Existing studies have primarily explored *external* enforcement of symbolic constraints ~~into-on~~ LMs. A common baseline is the *generate-and-validate* approach [21, 43], which repeatedly generates code and checks it with an external validator (e.g., a compiler) until a valid candidate is found. While this ensures constraint satisfaction, it is often inefficient due to a high rate of invalid generations.

To address this, *constrained decoding* techniques perform on-the-fly validation during token generation, discarding tokens that cannot lead to a valid complete program. This proactive strategy improves efficiency and has been applied to enforce various constraints, including syntactic correctness [33], type correctness [22], and vulnerability avoidance [29]. However, such token-level pruning can also distort the LM’s output distribution [25]. We compare with rejection sampling, grammar-constrained decoding (SynCode), completion-engine repair (Repilot), and iterative compiler-feedback repair in our evaluation (Tab. 5); ~~which is functionally equivalent to constrained decoding as both re-normalize the model’s output distribution over compilable programs.~~

Critically, both strategies enforce constraints *externally*. As discussed, externally enforced constraints do not modify the LM’s internal reasoning process. Consequently, the model’s internal distribution may become skewed, potentially assigning low probability to correct code snippets due to inaccurate assessments of their typability.

7.2 Internalizing Symbolic Constraints

Another line of research aims to expose the reasoning process of symbolic constraints to the LMs, thereby guiding them to learn to satisfy the constraints. By externalizing the complexity of constraint reasoning, this paradigm relieves LMs from implicitly inferring intricate symbolic rules from plain text. Offloading constraint learning to the representation allows LMs to allocate more capacity to other critical aspects, such as program semantics and algorithmic logic. This reallocation of modeling resources leads to improved generation quality that scales effectively with model size, mirroring the benefits observed in syntactic representations for grammatical learning [20].

Grammar-based encoding [30, 38, 42] represents one such approach, where programs are parsed into ASTs and serialized into sequences of grammar rules for training and inference. Grammar-based encoding can be viewed as a special case of our framework when grammar rules are expressed as CHCs. ~~Our approach generalizes this concept to arbitrary CHC-representable constraints~~ TyFlow extends this rule-based representation from syntactic structure to type-correctness proofs, with implementations and evaluations in Java and SuFu.

Tare [44] introduces a neural approach for learning Java’s type system in program repair, using specialized components to encode typing contexts and subtyping relations. While it predicts both code and types, Tare is specifically tailored to a subset of Java’s type system and operates through AST traversal without achieving full type explicitness, which is a key distinction from our work.

Refine4LLM [4] generates programs through sequential refinement rule applications, transforming specifications into concrete programs while ensuring syntactic and functional correctness. However, it requires training data containing refinement processes, which are scarce in practice, and is specific to a particular refinement calculus. In contrast, our approach utilizes readily available programs for training given a type checker, and ~~supports any constraints expressible as CHCs~~ the CHC formulation extends in principle to constraints beyond type correctness.

7.3 Type-Guided Program Synthesis

Our approach is inspired by type-guided program synthesis [11, 13, 24, 26], which employs synthesis rules to produce type-correct programs. These approaches also guarantee type correctness by construction; however, they are designed for classical enumerative or constraint-based synthesis algorithms, not for neural code generation.

In contrast, our work addresses three challenges that arise when adapting type-guided synthesis to LMs. First, existing synthesis systems do not provide a code representation suitable for LM training. We design synthesis decision sequences that can be derived from existing programs via type derivation trees, enabling the use of readily available codebases as training data. Second, existing systems do not enforce the structural correspondence between type derivations and synthesis derivations. We design our synthesis rules to satisfy Type Explicitness and Derivation Vicinality, establishing an isomorphism that makes type reasoning fully explicit in the generated sequences. Third, existing systems do not consider neural architectures for learning type systems. We propose a dual-encoding architecture that enables LMs to effectively learn and utilize type constraints during generation.

8 Conclusion

This paper presents TyFlow, a proof-guided approach that guarantees type correctness in code generation by constructing existential type correctness proofs through a synthesis process. Through a synthesis system isomorphic to the type system and a synthesis decision sequence representation, TyFlow ensures that constructing a synthesis derivation tree is equivalent to constructing an existential type correctness proof, thereby eliminating type errors during code generation, while significantly improving overall generation performance.

Beyond type correctness, TyFlow’s ~~foundation on CHC enables broader applications, including CHC foundation~~ suggests a route toward richer constraint families, such as safety verification and the generation of other structural data; instantiating and evaluating these extensions remains future work. Our core insight—that aligning program representations with underlying proof systems enhances ~~the~~ generation performance—opens promising directions ~~to in~~ other domains where structural constraints matter.

References

- [1] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, Sujan Kumar Gonugondla, Hantian Ding, Varun Kumar, Nathan Fulton, Arash Farahani, Siddhartha Jain, Robert Giaquinto, Haifeng Qian, Murali Krishna Ramanathan, and Ramesh Nallapati. 2023. Multi-lingual Evaluation of Code Generation Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=Bo7eeXm6An8>
- [2] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *CoRR* abs/2108.07732 (2021). arXiv:2108.07732 <https://arxiv.org/abs/2108.07732>
- [3] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2025. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 3639–3664. <https://aclanthology.org/2025.acl-long.183/>
- [4] Yufan Cai, Zhe Hou, David Sanán, Xiaokun Luan, Yun Lin, Jun Sun, and Jin Song Dong. 2025. Automated Program Refinement: Guide and Verify Code Large Language Model with Refinement Calculus. *Proc. ACM Program. Lang.*, 9, POPL (2025), 2057–2089. doi:10.1145/3704905
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish,

- Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [6] Alonzo Church. 1940. A Formulation of the Simple Theory of Types. *J. Symb. Log.* 5, 2 (1940), 56–68. doi:10.2307/2266170
- [7] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, Yan Liu, Enyu Zhou, Ming Zhang, Yuhao Zhou, Yueming Wu, Rui Zheng, Ming Wen, Rongxiang Weng, Jingang Wang, Xunliang Cai, Tao Gui, Xipeng Qiu, Qi Zhang, and Xuanjing Huang. 2024. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. *CoRR* abs/2407.06153 (2024). arXiv:2407.06153 doi:10.48550/ARXIV.2407.06153
- [8] Markus Freitag and Yaser Al-Onaizan. 2017. Beam Search Strategies for Neural Machine Translation. In *Proceedings of the First Workshop on Neural Machine Translation, NMT@ACL 2017, Vancouver, Canada, August 4, 2017*, Thang Luong, Alexandra Birch, Graham Neubig, and Andrew M. Finch (Eds.). Association for Computational Linguistics, 56–60. doi:10.18653/V1/W17-3207
- [9] Warren D. Goldfarb. 1981. The Undecidability of the Second-Order Unification Problem. *Theor. Comput. Sci.* 13, 2 (1981), 225–230. doi:10.1016/0304-3975(81)90040-2
- [10] Jiahsun Guo. 2025. Deep-Context-Awareness-Based LLM Code Generation and Accurate-Defect-Repair Integrated Architecture. *Journal of Artificial Intelligence Practice* 8 (2025), 125–133. doi:10.23977/jaip.2025.080217
- [11] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. 2020. Program synthesis by type-guided abstraction refinement. *Proc. ACM Program. Lang.* 4, POPL (2020), 12:1–12:28. doi:10.1145/3371080
- [12] Arie Gurfinkel and Nikolaj S. Björner. 2019. The Science, Art, and Magic of Constrained Horn Clauses. In *21st International Symposium on Symbolic and Numeric Algorithms for Scientific Computing, SYNASC 2019, Timisoara, Romania, September 4-7, 2019*. IEEE, 6–10. doi:10.1109/SYNASC49474.2019.00010
- [13] Sankha Narayan Guria, Jeffrey S. Foster, and David Van Horn. 2021. RbSyn: type- and effect-guided program synthesis. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 344–358. doi:10.1145/3453483.3454048
- [14] Siming Huang, Tianhao Cheng, J. K. Liu, Jiaran Hao, Liuyihan Song, Yang Xu, J. Yang, J. H. Liu, Chenchen Zhang, Linzheng Chai, Ruifeng Yuan, Haoxiang Zhang, Jie Fu, Qian Liu, Ge Zhang, Zili Wang, Yuan Qi, Yinghui Xu, and Wei Chu. 2024. OpenCoder: The Open Cookbook for Top-Tier Code Large Language Models. *CoRR* abs/2411.04905 (2024). arXiv:2411.04905 doi:10.48550/ARXIV.2411.04905
- [15] Ruyi Ji, Yuwei Zhao, Nadia Polikarpova, Yingfei Xiong, and Zhenjiang Hu. 2024. Superfusion: Eliminating Intermediate Data Structures via Inductive Synthesis. *Proc. ACM Program. Lang.* 8, PLDI (2024), 939–964. doi:10.1145/3656415
- [16] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *CoRR* abs/2406.00515 (2024). arXiv:2406.00515 doi:10.48550/ARXIV.2406.00515
- [17] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. 2024. Survey on Code Generation for Low resource and Domain Specific Programming Languages. *CoRR* abs/2410.03981 (2024). arXiv:2410.03981 doi:10.48550/ARXIV.2410.03981
- [18] Marie-Anne Lachaux, Baptiste Rozière, Lowik Chanasot, and Guillaume Lample. 2020. Unsupervised Translation of Programming Languages. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. <https://proceedings.neurips.cc/paper/2020/hash/f1d77dd9abf22dccc79b58dcd1ddcbad5-Abstract.html>
- [19] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [20] Qingyuan Liang, Zhao Zhang, Zeyu Sun, Zheng Lin, Qi Luo, Yueyi Xiao, Yizhou Chen, Yuqun Zhang, Haotian Zhang, Lu Zhang, Chenbin Chenbin, and Yingfei Xiong. 2025. Grammar-Based Code Representation: Is It a Worthy Pursuit for LLMs?. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 15640–15653. <https://aclanthology.org/2025.findings-acl.807/>
- [21] Fan Long and Martin C. Rinard. 2016. Automatic patch generation by learning correct code. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, Rastislav Bodík and Rupak Majumdar (Eds.). ACM, 298–312. doi:10.1145/2837614.2837617
- [22] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin T. Vechev. 2025. Type-Constrained Code Generation with Language Models. *CoRR* abs/2504.09246 (2025). arXiv:2504.09246 doi:10.48550/ARXIV.2504.09246
- [23] Nhan Nguyen and Sarah Nadi. 2022. An Empirical Evaluation of GitHub Copilot's Code Suggestions. In *19th IEEE/ACM International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23-24, 2022*. ACM, 1–5. doi:10.1145/3524842.3528470
- [24] Peter-Michael Osera and Steve Zdancewic. 2015. Type-and-example-directed program synthesis. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*, David Grove and Stephen M. Blackburn (Eds.). ACM, 619–630. doi:10.1145/2737924.2738007
- [25] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. Grammar-Aligned Decoding. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/2bdc2267c3d7d01523e2e17ac0a754f3-Abstract-Conference.html
- [26] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. 2016. Program synthesis from polymorphic refinement types. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2016, Santa Barbara, CA, USA, June 13-17, 2016*, Chandra Krantz and Emery D. Berger (Eds.). ACM, 522–538. doi:10.1145/2908080.2908093

- [27] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023, Miami, FL, USA, June 4-8, 2023*, Dawn Song, Michael Carbin, and Tianqi Chen (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html
- [28] John Alan Robinson. 1965. A Machine-Oriented Logic Based on the Resolution Principle. *J. ACM* 12, 1 (1965), 23–41. doi:10.1145/321250.321253
- [29] André Storhaug, Jingyue Li, and Tianyuan Hu. 2023. Efficient Avoidance of Vulnerabilities in Auto-completed Smart Contract Code Using Vulnerability-constrained Decoding. In *34th IEEE International Symposium on Software Reliability Engineering, ISSRE 2023, Florence, Italy, October 9-12, 2023*. IEEE, 683–693. doi:10.1109/ISSRE59848.2023.00035
- [30] Zeyu Sun, Qihao Zhu, Yingfei Xiong, Yican Sun, Lili Mou, and Lu Zhang. 2020. TreeGen: A Tree-Based Transformer Architecture for Code Generation. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 8984–8991. doi:10.1609/AAAI.V34I05.6430
- [31] Florian Tambon, Arghavan Moradi Dakhel, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Giuliano Antoniol. 2025. Bugs in large language models generated code: an empirical study. *Empir. Softw. Eng.* 30, 3 (2025), 65. doi:10.1007/S10664-025-10614-4
- [32] Sten-Åke Tärnlund. 1977. Horn Clause Computability. *BIT* 17, 2 (1977), 215–226.
- [33] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2025. SynCode: LLM Generation with Grammar Augmentation. *Trans. Mach. Learn. Res.* 2025 (2025). <https://openreview.net/forum?id=HiUZtgAPoH>
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [35] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, 8696–8708. doi:10.18653/V1/2021.EMNLP-MAIN.685
- [36] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 172–184. doi:10.1145/3611643.3616271
- [37] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *MAPS@PLDI 2022: 6th ACM SIGPLAN International Symposium on Machine Programming, San Diego, CA, USA, 13 June 2022*, Swarat Chaudhuri and Charles Sutton (Eds.). ACM, 1–10. doi:10.1145/3520312.3534862
- [38] Pengcheng Yin and Graham Neubig. 2017. A Syntactic Neural Model for General-Purpose Code Generation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, 440–450. doi:10.18653/V1/P17-1041
- [39] Biao Zhang, Paul Suganthan, Gaël Liu, Ilya Philippov, Sahil Dua, Ben Hora, Kat Black, Gus Martins, Omar Sanseviero, Shreya Pathak, Cassidy Hardin, Francesco Visin, Jiageng Zhang, Kathleen Kenealy, Qin Yin, Xiaodan Song, Olivier Lacombe, Armand Joulin, Tris Warkentin, and Adam Roberts. 2025. T5Gemma 2: Seeing, Reading, and Understanding Longer. arXiv:2512.14856 [cs.CL] <https://arxiv.org/abs/2512.14856>
- [40] Zhao Zhang, Qingyuan Liang, Zeyu Sun, Yizhou Chen, Guoqing Wang, Yican Sun, Lu Zhang, Ge Li, and Yingfei Xiong. 2025. GramTrans: A Better Code Representation Approach in Code Generation. arXiv:2510.02887 [cs.SE] <https://arxiv.org/abs/2510.02887>
- [41] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5673–5684.
- [42] Qihao Zhu, Qingyuan Liang, Zeyu Sun, Yingfei Xiong, Lu Zhang, and Shengyu Cheng. 2024. GrammarT5: Grammar-Integrated Pretrained Encoder-Decoder Neural Model for Code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 76:1–76:13. doi:10.1145/3597503.3639125
- [43] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*, Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.). ACM, 341–353. doi:10.1145/3468264.3468544
- [44] Qihao Zhu, Zeyu Sun, Wenjie Zhang, Yingfei Xiong, and Lu Zhang. 2023. Tare: Type-Aware Neural Program Repair. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1443–1455. doi:10.1109/ICSE48619.2023.00126

A Proofs for Some Lemmas Used in the Paper

A.1 Proof for Lemma 3.5

LEMMA A.1. *There is a bijection φ between typing rules and synthesis rules.*

PROOF. φ can be proven to be a bijection by showing that it is both surjective and injective.

- The translation function φ maps every typing rule to a synthesis rule, and synthesis rules are constructed exclusively in this manner; thus, φ is surjective.
- To prove that φ is injective, we define its inverse function ψ : given a synthesis rule in the form presented in Figure 8, ψ removes all substitutions σ_i , as well as the unification and assignment acquisition, and restores the terms $\bar{x}_0$ to the conclusion, recovering the typing rule as shown in Figure 7. Therefore, $\psi \circ \varphi = \varphi \circ \psi = id$. The existence of ψ demonstrates that φ is injective.

□

A.2 Proof for Lemma 4.1

LEMMA A.2. *For any synthesis judgment derived using a synthesis rule of the form in Figure 8, the composed substitution $\sigma_c = \sigma_n \dots \sigma_0 \sigma$ constitutes an assignment, in the sense that $\sigma_c(\bar{x}_i)$, $\sigma_c(\bar{x})$, and $\sigma_c(\bar{y})$ are all ground terms.*

PROOF. Induction on the structure of the synthesis derivation.

- **Base case:** The base case occurs when the synthesis rule has no subgoals. In this situation, the free variables $\bar{x}_f$ are precisely those appearing in the typing rule's conclusion, i.e., $FV(\bar{x}_f) = FV(\bar{x}_0)$. The unification step produces a substitution σ such that $\sigma(\bar{x}_0) = \sigma(\bar{x})$, and the assignment σ_0 obtained in this step covers all variables in $\sigma(\bar{x}_f)$.
Consequently, $\sigma_0 \sigma(\bar{x}_f)$ are ground terms, thus $\sigma_0 \sigma(\bar{x}_0) = \sigma_0 \sigma(\bar{x})$ are also ground terms, and so is $\sigma_0 \sigma(\bar{y})$ since $\bar{y}$ contains only variables that are in $\bar{x}_0$.
- **Inductive case:** The inductive case arises when the synthesis rule has one or more subgoals. By the induction hypothesis, for each i , the substitution σ_i is an assignment that covers all variables in $\sigma_{i-1} \dots \sigma_0 \sigma(\bar{x}_i)$, ensuring that $\sigma_i \dots \sigma_0 \sigma(\bar{x}_i)$ are ground terms, and therefore so is $\sigma_c(\bar{x}_i)$ for all $i \geq 1$.
We similarly observe that $\sigma_0 \sigma$ covers all variables in $\bar{x}_f$ so σ_c must also cover $\bar{x}_f$ and thus $\bar{x}_0$ ($\bar{x}_f$ contains all variables in $\bar{x}_0$ besides those already instantiated in $\bar{x}_i$). We finally have $\sigma_c(\bar{x}) = \sigma_n \dots \sigma_0 \sigma(\bar{x}) = \sigma_n \dots \sigma_0 \sigma(\bar{x}_0) = \sigma_c(\bar{x}_0)$ is a ground term, and so is $\sigma_c(\bar{y})$, since $\bar{y}$ contains only variables that are in $\bigcup_{i=0}^n \bar{x}_i$.

□

A.3 Proof for Lemma 4.2

LEMMA A.3. *Let T_1 be a well-formed type derivation tree with root node labeled $P(\bar{x}_c)$, annotated with typing rule R , and let T_2 be its corresponding synthesis derivation tree constructed via the translation, whose root node is labeled with $P(\bar{x}) \rightsquigarrow \theta$, where θ is the assignment synthesized by Algorithm 1. If $\bar{x}$ is unifiable with $\bar{x}_c$, then in the application of the synthesis rule that derives $P(\bar{x}) \rightsquigarrow \theta$, (1) the unification step, (2) the acquisition for free variables, and (3) the constraint checking step will always succeed, and (4) the synthesized assignment θ makes $P(\bar{x})$ concretized to $P(\theta(\bar{x})) = P(\bar{x}_c)$.*

PROOF. Induction on the size of the type derivation tree T_1 . Suppose the typing rule R has the form shown in Figure 7, and its corresponding synthesis rule R' has the form shown in Figure 8.

- **Base case:** The base case is when the type derivation tree T_1 has only one node, which means the typing rule R has no premises. In this case, since $\bar{x}$ is unifiable with $\bar{x}_c$ and in the typing rule R , $\bar{x}_c$ is the concretization of $\bar{x}_0$, the unification and acquisition steps will succeed, yielding a substitution σ such that $\sigma(\bar{x}_0) = \sigma(\bar{x})$ and $\theta'\sigma(\bar{x}_0) = \bar{x}_c$ for some assignment θ' , which can be used to obtain $\sigma_0 = \theta'|_{\text{FV}(\sigma(\bar{x}_f))} = \theta'|_{\text{FV}(\sigma(\bar{x}_0))}$. By [lemma 4.1](#), $\sigma_0\sigma$ is an assignment that covers all variables in $\bar{x}$, $\bar{x}_0$ and $\bar{y}$, and $\sigma_0\sigma(\bar{x}_0) = \theta'\sigma(\bar{x}_0) = \bar{x}_c$, so if we set $\theta = \sigma_0\sigma|_{\text{FV}(\bar{x})}$ we have $\theta(\bar{x}) = \sigma_0\sigma(\bar{x}) = \sigma_0\sigma(\bar{x}_0) = \bar{x}_c$. The constraint checking step is also satisfied because $\text{FV}(\bar{y}) \subseteq \text{FV}(\bar{x}_0)$ and $\sigma_0\sigma(\bar{y})$ is the same as the verified concretized constraint in the type derivation tree.
- **Inductive case:** If the typing rule R has a series of [of](#) premises $P_1(\bar{x}_1), \dots, P_n(\bar{x}_n)$, then the synthesis rule R' has a series of subgoals $P_1(\sigma_0\sigma(\bar{x}_1)) \rightsquigarrow \sigma_1, \dots, P_n(\sigma_{n-1} \dots \sigma_0\sigma(\bar{x}_n)) \rightsquigarrow \sigma_n$.
[The same as](#) [As in](#) the base case, the unification and acquisition steps will succeed, yielding a substitution σ such that $\sigma(\bar{x}_0) = \sigma(\bar{x})$ and $\theta'\sigma(\bar{x}_0) = \bar{x}_c$ for some assignment θ' , and ~~the~~ $\sigma_0 = \theta'|_{\text{FV}(\bar{x}_f)}$.
 In the type derivation tree, suppose the i -th child node of the root node is labeled with $P_i(\bar{x}_{ic})$, then rule R can be instantiated with some assignment θ_c such that $\theta_c(\bar{x}_i) = \bar{x}_{ic}$ for all i ($\bar{x}_{0c}$ is just $\bar{x}_c$). We use the notation $\sigma_i \trianglelefteq \sigma_j$ to denote that σ_j factors through σ_i , meaning that $\sigma_j = \sigma'\sigma_i$ for some substitution σ' . Following the derivation of σ_0 , we know that $\theta'\sigma(\bar{x}_0) = \bar{x}_c$ and $\sigma_0 = \theta'|_{\text{FV}(\sigma(\bar{x}_f))} \trianglelefteq \theta'|_{\text{FV}(\sigma(\bar{x}_0))}$, thus $(\sigma_0\sigma|_{\text{FV}(\bar{x}_0)}) \trianglelefteq (\theta'\sigma|_{\text{FV}(\bar{x}_0)}) \trianglelefteq \theta_c$ holds (θ_c concretizes $\bar{x}_0$) and $\sigma_0\sigma$ ranges over only variables in $\bar{x}_0$ and $\bar{x}$.
 Since $\bar{x}_1$ has no common variables with $\bar{x}$, $\sigma_0\sigma(\bar{x}_1) = (\sigma_0\sigma|_{\text{FV}(\bar{x}_0)})(\bar{x}_1)$, and $\theta_c(\bar{x}_1) = \bar{x}_{1c}$, ~~hence~~ $\sigma_0\sigma(\bar{x}_1)$ is unifiable with $\bar{x}_{1c}$. Thus we can use the induction hypothesis to ~~synthesize~~ [synthesize](#) $P_1(\sigma_0\sigma(\bar{x}_1)) \rightsquigarrow \sigma_1$, where $\sigma_1\sigma_0\sigma(\bar{x}_1) = \bar{x}_{1c}$.
~~Then we continue.~~ [Then, note](#) that $\sigma_1\sigma_0\sigma(\bar{x}_1) = \theta_c(\bar{x}_1)$, and ~~$(\sigma_1\sigma_0\sigma|_{\text{FV}(\bar{x}_0, \bar{x}_1)}) \trianglelefteq \theta'$~~ ~~$(\sigma_1\sigma_0\sigma|_{\text{FV}(\bar{x}_0, \bar{x}_1)}) \trianglelefteq \theta'$~~ holds, and then ~~$\sigma_1\sigma_0\sigma(\bar{x}_2) = (\sigma_1\sigma_0\sigma|_{\text{FV}(\bar{x}_0, \bar{x}_1)})(\bar{x}_2)$~~ ~~$\sigma_1\sigma_0\sigma(\bar{x}_2) = (\sigma_1\sigma_0\sigma|_{\text{FV}(\bar{x}_0, \bar{x}_1)})(\bar{x}_2)$~~ is unifiable with $\bar{x}_{2c}$, and we can apply the induction hypothesis again and again until all assignments to subgoals are synthesized.
 Finally, we can collect all the assignments $\sigma_n, \dots, \sigma_1, \sigma_0$ and compose them to form $\sigma_c = \sigma_n \dots \sigma_0\sigma$, which is an assignment and covers all variables in $\bar{x}_i$, $\bar{x}$ and $\bar{y}$ by [Lemma 4.1](#). $\sigma_c(\bar{x}_i) = \bar{x}_{ic} = \theta_c(\bar{x}_i)$ for all i , so $\theta_c \trianglelefteq \sigma_c$ since σ_c also ranges over $\text{FV}(\bar{x})$. We have $\sigma_c(\bar{y}) = \theta_c(\bar{y})$, which instantiates the constraint $\phi(\bar{y})$ to be satisfied as in the type derivation tree. [And let Here](#), $\theta(\bar{x}) = \sigma_c|_{\text{FV}(\bar{x})}$ concretizes the synthesis goal $P(\bar{x})$ to the ground typing judgment $P(\bar{x}_c)$ ($\theta(\bar{x}) = \sigma_n \dots \sigma_0\sigma(\bar{x}) = \sigma_n \dots \sigma_0\sigma(\bar{x}_0) = \sigma_c(\bar{x}_0) = \bar{x}_c$), which is the same as the root node of the type derivation tree.

□

A.4 Proof for Lemma 4.4

LEMMA A.4. For any synthesis derivation tree T_2 , there exists a type derivation tree T_1 , $T_1 \cong T_2$.

PROOF. Following the translation from synthesis derivation trees to the type derivation trees detailed in [Sec. 4.3](#), the root and child order preservation is easily satisfied. Then we can prove the label correspondence by induction on the size of the synthesis derivation tree T_2 (suppose T_2 's top-level synthesis rule has the form in [Fig. 8](#)).

- **Base case:** The base case occurs when the synthesis derivation tree T_2 has only one node, whose synthesis rule has no premises. By the described translation, the node in the type derivation tree T_1 is annotated with the typing judgment $P(\theta(\bar{x}))$, if θ is the synthesized assignment. In this case, the label correspondence is clearly satisfied.

- **Inductive case:** The inductive case arises when the synthesis rule has one or more subgoals. By the induction hypothesis, for the i -th subgoal $P_i(\sigma_{i-1} \dots \sigma_0 \sigma(\bar{x}_i)) \leadsto \sigma_i$, we have constructed a sub-type derivation tree type derivation subtree whose root node is labeled with the typing judgement $P_i(\bar{x}_{ic})$, where $\bar{x}_{ic} = \sigma_i \dots \sigma_0 \sigma(\bar{x}_i)$. Let $\theta_c = \sigma_i \dots \sigma_0 |_{FV(\bar{x}_0 \cup \dots \cup \bar{x}_n)}$, $\theta_c = \sigma_n \dots \sigma_0 |_{FV(\bar{x}_0 \cup \dots \cup \bar{x}_n)}$, then we have $\theta_c(\bar{x}_i) = \sigma_i \dots \sigma_0 \sigma(\bar{x}_i) = \bar{x}_{ic}$ for all $i \geq 1$. For the conclusion part, $\theta_c(\bar{x}_0) = \sigma_0 \sigma(\bar{x}_0) = \sigma_0 \sigma(\bar{x}) = \sigma_n \dots \sigma_0 \sigma(\bar{x}) = \theta(\bar{x})$. Thus, under the assignment θ_c , the corresponding typing rule, along with the translated sub-type derivation tree type derivation subtrees, forms a valid type derivation tree T_1 whose root node is labeled with $P(\theta(\bar{x}))$.

□

B Additional Information about Benchmarks

B.1 Java Benchmark

We have carefully designed a subset of the Java programming language that encompasses the fundamental syntactic constructs, including The implemented Java subset supports classes, methods, variables, arrays, generic types, operators, and control flow statements. ~~While excluding advanced Java features such as~~; it omits multithreading, lambda expressions, and complex reflection mechanisms.

The Java subset retains sufficient expressiveness to represent core imperative programming paradigms. Our evaluation shows that it covers approximately 78% of the Java programs in the original MBJP dataset, which is the Java version of the MBPP dataset and available at [amazon-science/mxeval](https://amazon-science.github.io/mxeval/). Evaluation uses only tasks built from constructs this grammar supports, so the reported results apply to this supported subset. The complete grammar specification is presented below:

Java Subset Language Grammar	
<program>	<pre> := <program> <program> <modifier> <type> <identifier> ; <modifier> <type> <identifier> = <term> ; <modifier> <type> <identifier> (<stmt>) { <stmt> } <identifier> (<stmt>) { <stmt> } <stmt> class <identifier> { <program> } </pre>
<stmt>	<pre> := empty <type> <identifier> = <term>; <type> <identifier>; if (<term>) { <stmt> } if (<term>) { <stmt> } else { <stmt> } while (<term>) { <stmt> } do { <stmt> } while (<term>) for (<stmt> ; <term> ; <term>) { <stmt> } for (<type> <identifier> : <term>) { <stmt> } return <term>; continue; break; </pre>

```

| <term>;
| switch ( <term> ) { <stmt> }
| case <term> : <stmt>
| <stmt> <stmt>
<term> := <identifier>
| <integer>
| <float>
| <character>
| <string>
| true
| false
| null
| <term> = <term>
| ( <type> ) <term>
| <term> instanceof <type>
| <term> ? <term> : <term>
| <term> . <identifier>
| <term> [ <term> ]
| new <type> ( <term> )
| new <type>
| new <term> [ <term> ]
| <term> . <identifier> ( <term> )
| <identifier> ( <term> )
| <type>
| [ <term_list> ]
| { <term_list> }
| ( <term> )
| <term> + <term>
| <term> - <term>
| <term> * <term>
| <term> / <term>
| <term> % <term>
| - <term>
| <term> << <term>
| <term> >> <term>
| <term> & <term>
| <term> | <term>
| <term> ^ <term>
| ~ <term>
| <term> ++
| <term> --
| ++ <term>
| -- <term>

```

	<term> == <term>
	<term> != <term>
	<term> < <term>
	<term> <= <term>
	<term> > <term>
	<term> >= <term>
	<term> && <term>
	<term> <term>
	! <term>
<type>	:= int
	float
	boolean
	char
	String
	null
	<type> []
	<identifier> <>
	<identifier> < <type> >
	<identifier> < <type> , <type> >
	<type> -> <type>
	<identifier>
	void
<term_list>	:= <term> , <term_list>
	<term>
	empty
<modifier>	:= public private protected static final abstract
<identifier>	:= [a-zA-Z_][a-zA-Z0-9_]*
<integer>	:= [0-9]+
<float>	:= [0-9]+.[0-9]+
<character>	:= '['^']'
<string>	:= ".*"

Every Java task is presented to the model as a single prompt with a fixed template: the import statements, a class skeleton that declares the target method, and a Javadoc comment carrying the natural-language description; the model completes the body of the declared method. The MBJP and HumanEval-Java prompts additionally embed executable example calls with their expected outputs in the Javadoc. The following prompt from MBJP, reproduced in the original mxeval release format, illustrates the template:

Example Java Prompt (MBJP)

```
import java.io.*;
import java.lang.*;
import java.util.*;
```

```
import java.math.*;

class IsPolite {
    /**
     * * Write a function to find nth polite number.
     *
     * * > isPolite(7)
     * * 11
     * * > isPolite(4)
     * * 7
     * * > isPolite(9)
     * * 13
     */
    public static int isPolite(int n) {
```

B.2 HumanEval-Java and TransCoder-GFG

HumanEval-Java is the Java port of HumanEval [5]: short Python-style algorithm snippets (string/list manipulation, digit arithmetic, parity and divisibility tests, a few short geometry/number-theory problems) transcribed into a single Java method per task. **TransCoder-GFG** reuses the GeeksforGeeks Java programs released with TransCoder [18]: one- or two-sentence specifications (numeric, string, or array algorithms; standard-library types only) paired with a self-contained Java method. Tab. 7 summarizes the three Java evaluation splits used in this paper.

Table 7. Test-split statistics of the three Java benchmarks.

Benchmark (train / test)	prompt (words)	gold LoC
MBJP (608 / 67) [1]	63 (46–137)	27 (19–43)
HumanEval-Java (146 / 16) [5]	88 (53–135)	12 (5–53)
TransCoder-GFG (414 / 103) [18]	65 (55–105)	12 (3–31)

Cell entries are *median (range)*; prompt length counts whitespace-separated words on the full prompt (imports + class skeleton + javadoc); gold LoC counts non-blank lines of the canonical reference solution; difficulty labels for TransCoder-GFG (easy / medium / hard) are 53 / 49 / 1.

HumanEval-Java and TransCoder-GFG problems are short single-method snippets (median 12 LoC), whereas MBJP is the densest and most example-rich benchmark (median 27 LoC, prompt with three executable example cases and a gold solution about twice as long as the additional benchmarks).

HumanEval-Java and TransCoder-GFG baselines. The T5Gemma2-2B baselines for the two additional benchmarks are fine-tuned with learning rate 10^{-5} for 30 passes, selecting the checkpoint on validation holdouts drawn from the respective training splits (32 and 60 tasks, never in any training data). Each selected configuration was then evaluated once on the fixed test split, with the standard beam-10 decoding protocol and no few-shot prompting. Per-candidate results are part of the artifact package.

B.3 SuFu Benchmark

The SuFu dataset is sourced from the GitHub repository accompanying the SuperFusion research paper ([jiry17/SuFu](https://github.com/jiry17/SuFu)). SuperFusion is an advanced program optimization tool that processes SuFu programs as input and applies a series of transformations to improve computational efficiency. The SuFu language features a type system that explicitly specifies those intermediate data structures to be optimized, and all code fragments operating on these data structures. The SuFu programs in this dataset are drawn from two primary lines of prior work. The first source consists of fusion tasks primarily collected from classical textbooks such as Introduction to Algorithms, while the second originates from research on program restructuring, where the goal is to transform a reference program into specific target forms such as divide-and-conquer or single-pass. Each program is supplemented with a natural language description. These descriptions were first generated by the GPT-o3-mini model and then carefully reviewed and edited by human experts to correct inaccuracies and improve clarity. Each task prompt concatenates this description with the relevant inductive definitions and library functions, which are given as a program prefix; the remaining program, i.e., the target function and the main expression, is the part to be synthesized. The following prompt from the test split illustrates the format:

Example SuFu Prompt

Write a sufu program that, given a custom list type (ending with elt), checks if the list is sorted in non-decreasing order. If it is sorted, it computes the minimum element of the list; otherwise, it returns 0.

```
Inductive List = elt Int | cons {Int, List};
is_sorted = let aux = (fix (
  \f: Int -> List -> Bool. \pre: Int. \xs: List.
  match xs with
  | elt x -> <= pre x
  | cons {h, t} -> and (<= pre h) (f h t)
  end
)) in
  \xs: List.
  match xs with
  | elt x -> true
  | cons {h, t} -> aux h t
  end;
min = \a: Int. \b: Int.
  if (> a b) then b
  else a;
spec = fix (
  \f: List -> Int. \xs: List.
  match xs with
  | elt w -> w
  | cons {h, t} -> min h (f t)
  end
);
```

A key issue with the SuFu language is that when an algorithm description is provided, the initial SuFu program still needs to be manually constructed. However, manually constructing SuFu programs from algorithmic descriptions is highly challenging and requires considerable domain expertise in functional programming. Directly prompting language models to generate SuFu code from natural language descriptions often results in programs that are syntactically or semantically flawed, typically containing multiple type errors.

The following is an example of ~~generated SuFu programs~~ a generated SuFu program with type errors:

Natural Language Prompt

Write a function for the mts algorithm, which finds the maximum sum among all possible tail lists of the input list.

```
tails :: List -> Packed NList
tails Nil = let ts = NCons(Nil, NNil) in
  rewrite (label ts)
tails Cons(_, t)@xs = let ts = tails t in
  rewrite (label NCons(xs, ts))
  // there is a type error, `ts` should be `unlabel ts`
mts_label xs = let ts = tails xs in
  rewrite (maximum (map sum (unlabel ts)))
```

The complete SuFu grammar specification is as follows:

SuFu Language Grammar

```
<program>    := <program> <program>
              | <command>
<command>    := Inductive <Name> = <name> <type>
              | ...
              | <name> <type>;
              | <Name> = <type>;
              | <name> = <term>;
<type>       := {<type>, ..., <type>}
              | <type> -> <type>
              | Reframe <type>
              | Unit
              | Int
              | Bool
              | <Name>
<term>       := unit
              | <integer>
              | true
              | false
              | <name>
              | + | - | * | /
```


	< <= > >= ==
	and or not
	<term> <term>
	\<name>:<type>. <term>
	fix <term>
	let <name> = <term> in <term>
	if <term> then <term> else <term>
	{<term>, ..., <term>}
	<term>.<integer>
	match <term> with <pattern> -> <term>
	...
	<pattern> -> <term>
	end
	rewrite <term>
	label <term>
	unlabel <term>
<pattern>	:= _
	<name>
	<name> <pattern>
	{<pattern>, ..., <pattern>}
<Name>	:= [A-Z][a-zA-Z0-9_]*
<name>	:= [a-z][a-zA-Z0-9_]*
<integer>	:= [0-9]+

C Failure Analysis

This section classifies the failing tasks of the Java evaluation and presents representative cases. For each benchmark and model, every test task is assigned to one of four mutually exclusive categories: **Solved@1** (the top-1 candidate passes all tests); **Ranking failure** (some candidate in the top ten passes, but not the top-1 one); **All-invalid** (no candidate passes and none of the ten candidates compiles); and **Well-typed but wrong** (no candidate passes, but at least one candidate compiles—the failure is semantic, not syntactic).

Table 8. Task-Level Failure Taxonomy on the Three Java Test Sets

Benchmark	Model	Solved@1	Ranking	All-invalid	Well-typed wrong	Total
MBJP (67)	T5Gemma2-2B	9	13	4	41	67
	TyFlow-2B	17	12	0	38	67
HumanEval-Java (16)	T5Gemma2-2B	5	1	0	10	16
	TyFlow-2B	8	1	0	7	16
TransCoder-GFG (103)	T5Gemma2-2B	20	14	2	67	103
	TyFlow-2B	31	17	2	53	103

Counts in Tab. 8 are derived from per-candidate statuses of the frozen runs; timeouts and unfilled slots count as invalid, and the two TransCoder-GFG all-invalid TyFlow-2B tasks include the documented budget-exhaustion (fail-closed) task. On MBJP, the baseline produces no compilable candidate for four tasks, while TyFlow produces at least one for every task. On HumanEval-Java, neither model has a task with no compilable candidate; on TransCoder-GFG, both models have two such tasks. Most tasks not solved by TyFlow at pass@1 have at least one compilable candidate but no candidate passing all tests. On MBJP, for example, this accounts for 38 of the 50 tasks not solved at pass@1; it is also the largest remaining failure category on the other two benchmarks. In other cases, a correct candidate is generated but not ranked first: this occurs on 12 MBJP tasks, one HumanEval-Java task, and 17 TransCoder-GFG tasks. These tasks count as successes at pass@10 but not at pass@1.

C.1 Representative Case: Invalid Generation (T5Gemma2-2B)

MBJP task 5 (*tupleStrInt*, converting a string tuple to an integer list) is unsolved by the ordinary baseline with all ten candidates failing to compile. The top-1 candidate omits the opening brace of the loop body:

```
public static List<Integer> tupleStrInt(String testStr) {
    int count = 0;
    List<Integer> ret = new ArrayList<>();
    for (String s : testStr.split("[, ]+"))
        count = count << 1;
        ret.add(Integer.valueOf(s.trim()));
    }
    return ret;
}
```

Without the brace, the loop body is a single statement, the closing brace terminates the method early, and javac reports illegal start of type at the return statement. The model produces plausible token-level continuations, but nothing prevents such structural corruption.

C.2 Representative Case: Well-Typed but Wrong (TyFlow-2B)

MBJP task 0 (*oddPosition*, checking that every odd index holds an odd number) is unsolved by TyFlow-2B, yet its top-1 candidate compiles and type-checks:

```
public static boolean oddPosition(List<Integer> nums) {
    boolean res = false;
    for (int i = 0; i < nums.size(); i++) {
        if (i % 2 == 0) {
            res = false;
        }
    }
    return res;
}
```

The candidate has the right shape—a loop over indices with a modulo test—but the body never inspects `nums.get(i)` and never assigns `true`; the method returns `false` for every input, while the tests expect `true` for two of the three cases. The candidate satisfies the implemented typing rules, but its behavior does not match the task specification. This case illustrates a general logic error rather than a failure confined to edge cases; the taxonomy does not separately count edge-case failures.

C.3 SuFu

On SuFu, TyFlow eliminates compilation errors entirely (CER = 0 in Tab. 2), so every unsolved task falls into the well-typed-but-wrong or ranking category.

D Statistical Significance Analysis

This section complements the point estimates in Tab. 2 with a paired significance analysis per language. It addresses whether TyFlow’s improvements over its base models are statistically distinguishable on Java and on SuFu.

We evaluate statistical significance per language: for each of Java and SuFu, the per-task paired differences of the 220M and 2B comparisons are pooled and tested jointly with a two-sided sign-flip randomization test (200,000 sign flips). Under the null hypothesis, the sign of each paired difference is exchangeable, and the flips automatically account for the correlation induced by tasks shared between the two scales. Pooling the scales per language is needed because each individual language–scale block (58–186 tasks) is too small for a well-powered separate test. 95% CIs for the combined mean paired difference come from a task-level cluster bootstrap (20,000 resamples), which preserves the same within-task correlation. Table 9 reports the results: all eight combined differences favor TyFlow, with 95% CIs excluding zero.

Table 9. Combined Cross-Scale Sign-Flip Tests (per language: 220M + 2B paired differences pooled)

Language	Metric	Mean paired diff.	95% CI (cluster bootstrap)	Sign-flip p -value
SuFu (58 tasks $\times$ 2 scales)	pass@1	+12.1 pp	[+2.6, +21.6]	2.50×10^{-2}
	pass@10	+16.4 pp	[+6.0, +26.7]	4.2×10^{-3}
	FSP	−1.63 positions	[−2.58, −0.70]	1.0×10^{-3}
	CER	−0.79	[−0.83, −0.75]	$< 1 \times 10^{-4}$
Java (67 tasks $\times$ 2 scales + 119 $\times$ 1)	pass@1	+9.1 pp	[+3.2, +15.0]	3.6×10^{-3}
	pass@10	+11.5 pp	[+5.5, +17.4]	4.0×10^{-4}
	FSP	−1.00 positions	[−1.54, −0.46]	3.0×10^{-4}
	CER	−0.23	[−0.27, −0.19]	$< 1 \times 10^{-4}$

Mean paired difference: pass@1/pass@10 in percentage points of tasks and CER in fraction of candidates per task (negative = fewer errors); FSP in positions of the first successful candidate. CIs resample tasks, keeping the 220M and 2B differences of a shared task together.